funTDA and ML

Xiaoyu Qi & 25232053

The R workflow consists of two analytical modules: functional topological data analysis (funTDA) and machine learning (ML).

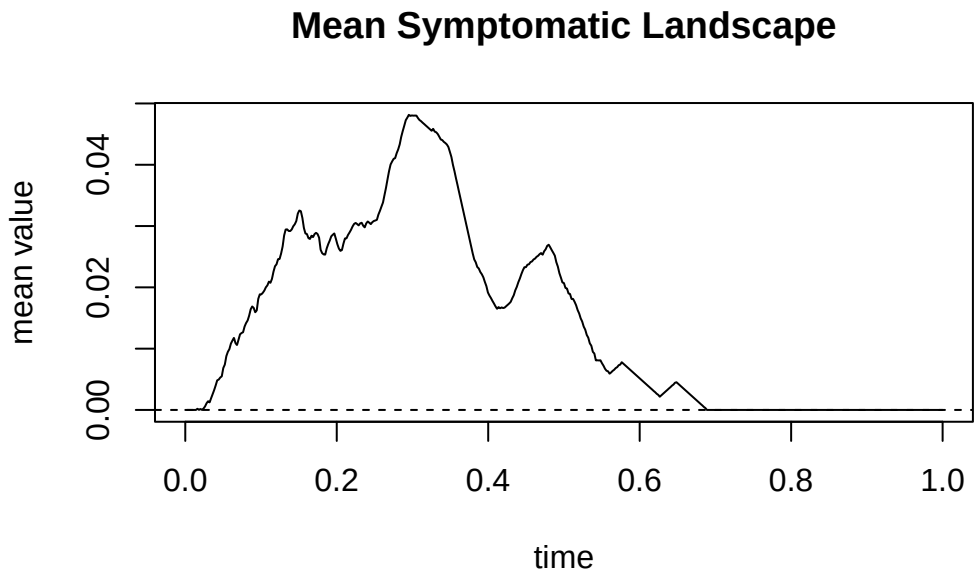
funTDA

funTDA Code Source: Catherine Higgins (2025), used under MIT License.

```
# ==================================================================
# Load libraries
# ==================================================================
library(TDA)
library(fda)
library(ggplot2)
library(ggrepel)
library(patchwork)
set.seed(123)

# ==================================================================
# Path to files
# ==================================================================
ph_folder <- "../../output/PH"
files <- list.files(ph_folder, pattern = "_PH.csv$", full.names = TRUE)

#extract the numeric ID from filename
get_number <- function(fname){
 as.numeric(sub(".*adjmatrix([0-9]+)_PH.*", "\\1", fname))
}

#order files numerically
file_numbers <- sapply(files, get_number)
files <- files[order(file_numbers)]
```

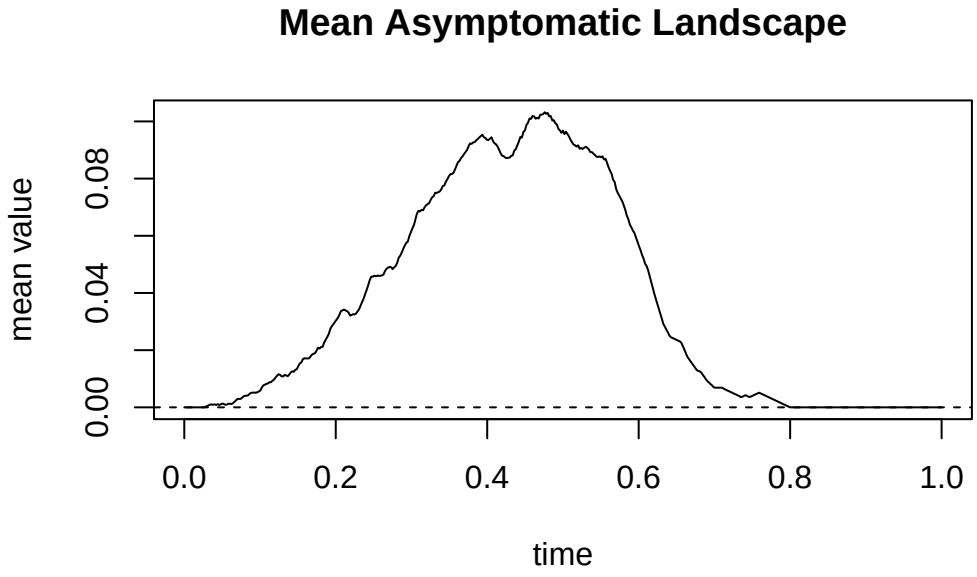
```r
# ================================================================
# Compute persistence landscapes
# ================================================================
DataSymp <- c()
DataAsymp <- c()
tseq <- seq(0, 1, length.out = 500)

read_landscape <- function(file_path, dim = 1, tseq = tseq) {
  PH <- read.csv(file_path)
  colnames(PH) <- c("Birth", "Death", "dimension")
  PH <- PH[c("dimension", "Birth", "Death")]
  PH <- as.matrix(PH)
  x <- landscape(PH, dimension = dim, KK = 1, tseq)
  return(x)
}

for(f in files){
  x1 <- read_landscape(f, dim = 1, tseq = tseq)

  # detect group from filename
  fname <- basename(f)
  if(grepl("^Asymptomatic", fname)){
    DataAsymp <- cbind(x1, DataAsymp)
  } else if(grepl("^Symptomatic", fname)){
    DataSymp <- cbind(x1, DataSymp)
  } else {
    warning(paste("File not assigned to group:", fname))
  }
}


DataSymp <- t(DataSymp)
DataAsymp <- t(DataAsymp)
DataBoth <- rbind(DataAsymp, DataSymp)

# ================================================================
# Create FD objects
# ================================================================
create_fd <- function(Data){
  x <- seq(0, 1, length.out = ncol(Data))
  basis <- create.bspline.basis(c(0,1), nbasis = ncol(Data)-1, norder = 2)
  fdobj <- smooth.basis(x, t(Data), basis)
```

```
  return(fdobj$fd)
}

fdSymp <- create_fd(DataSymp)
fdAsymp <- create_fd(DataAsymp)
fdBoth <- create_fd(DataBoth)

plot(mean.fd(fdSymp), main="Mean Symptomatic Landscape")
```




```
[1] "done"
```

```
plot(mean.fd(fdAsymp), main="Mean Asymptomatic Landscape")
```




```
[1] "done"
```

```
# ================================================================
# Functional PCA
# ================================================================

n_pc_max <- nrow(DataBoth) - 1   # 16
pca <- pca.fd(fdBoth, nharm = n_pc_max)

m <- c("A1","A2","A3","A4","A5","A6","A7","A8",
       "S1","S2","S3","S4","S5","S6","S7","S8","S9")
col.group <- c(rep("black", 8), rep("blue", 9))
group <- c(rep("Asymptomatic", 8), rep("Symptomatic", 9))

varprop <- round(pca$varprop * 100, 1)          # 16
cum_var <- cumsum(varprop)

# Print the variance and cumulative variance for each PC
for (i in 1:length(varprop)) {
  cat(sprintf("PC%d explains %.1f%% variance (cumulative %.1f%%)\n",
              i, varprop[i], cum_var[i]))
}
```

```
PC1 explains 63.5% variance (cumulative 63.5%)
PC2 explains 20.0% variance (cumulative 83.5%)
```

```
PC3 explains 6.7% variance (cumulative 90.2%)
PC4 explains 3.3% variance (cumulative 93.5%)
PC5 explains 2.2% variance (cumulative 95.7%)
PC6 explains 1.2% variance (cumulative 96.9%)
PC7 explains 0.8% variance (cumulative 97.7%)
PC8 explains 0.7% variance (cumulative 98.4%)
PC9 explains 0.5% variance (cumulative 98.9%)
PC10 explains 0.3% variance (cumulative 99.2%)
PC11 explains 0.2% variance (cumulative 99.4%)
PC12 explains 0.2% variance (cumulative 99.6%)
PC13 explains 0.2% variance (cumulative 99.8%)
PC14 explains 0.1% variance (cumulative 99.9%)
PC15 explains 0.1% variance (cumulative 100.0%)
PC16 explains 0.0% variance (cumulative 100.0%)
```

```r
threshold <- 0.95   # 95 % PC Number = 5
n_pc_keep <- which(cum_var >= threshold * 100)[1]   # PC number
if (n_pc_keep < 2) warning("If there are fewer than 2 PCs, 2D plotting will fail. Please lowe
cat(sprintf("Retain %d PCs to explain at least %.1f%% variance.\n",
            n_pc_keep, cum_var[n_pc_keep]))
```

```
Retain 5 PCs to explain at least 95.7% variance.
```

```r
fpc_table <- data.frame(
  PC = 1:n_pc_keep,
  Variance = round(varprop[1:n_pc_keep], 1),
  Cumulative = round(cum_var[1:n_pc_keep], 1)
)

colnames(fpc_table) <- c("PC", "Variance (%)", "Cumulative (%)")

print(fpc_table)
```

```
  PC Variance (%) Cumulative (%)
1  1         63.5           63.5
2  2         20.0           83.5
3  3          6.7           90.2
4  4          3.3           93.5
5  5          2.2           95.7
```

```r
pca_scores <- pca$scores[, 1:n_pc_keep]   # Preserve dynamically selected PCs

pca_df <- data.frame(
  PC1 = pca$scores[, 1],
  PC2 = pca$scores[, 2],
  ID = m,
  Group = group
)

  pca_plot <- ggplot(pca_df, aes(x = PC1, y = PC2, color = Group, label = ID)) +
    geom_hline(yintercept = 0, linetype = "dashed", color = "black") +
    geom_vline(xintercept = 0, linetype = "dashed", color = "black") +
    geom_point(size = 4, alpha = 0.9) +
    ggrepel::geom_text_repel(size = 4, show.legend = FALSE, max.overlaps = Inf) +
    scale_color_manual(values = c("Asymptomatic" = "black", "Symptomatic" = "blue")) +
    labs(
      x = "PC1 Score",
      y = "PC2 Score",
      title = ""
    ) +
    coord_fixed() +
    theme_classic(base_size = 16) +
    theme(
      legend.position = ifelse(T, "right", "none"),
      legend.title = element_blank(),
      plot.title = element_text(hjust = 0.5, face = "bold")
    )

  print(pca$scores)
```

```
             [,1]          [,2]          [,3]          [,4]          [,5]
 [1,]  0.00852785 -0.0325549057  0.0095234436  0.0004460247 -0.0060167312
 [2,]  0.01875517 -0.0008015288  0.0058615171  0.0059652832 -0.0033478719
 [3,]  0.01743687  0.0114537304 -0.0062200133  0.0045453282  0.0012269731
 [4,]  0.02711670  0.0019350019  0.0003426474 -0.0027638882 -0.0024553874
 [5,]  0.02193849  0.0084664363 -0.0038669797  0.0042098060 -0.0007040496
 [6,]  0.01941269  0.0088981626  0.0037899939 -0.0032306927 -0.0050325406
 [7,]  0.03048737 -0.0157788884 -0.0043821009 -0.0105904705  0.0081089152
 [8,]  0.01483932  0.0119973255 -0.0028872593  0.0004915137 -0.0002331193
 [9,]  0.01057673 -0.0068572355 -0.0018682034  0.0094634080  0.0062437623
[10,] -0.02420064 -0.0087968991 -0.0074041787 -0.0001694307  0.0010943843
[11,] -0.01730829  0.0073222808  0.0037815051 -0.0024414608  0.0030366465
```

```
[12,] -0.02887330 -0.0072504245 -0.0064305878  0.0055589641 -0.0012494260
[13,] -0.02904847 -0.0098757796 -0.0092210463 -0.0011532250 -0.0011718596
[14,] -0.01660377  0.0067107820  0.0112716733  0.0007972986  0.0052429104
[15,] -0.01858342  0.0099368055  0.0092404603 -0.0030562807  0.0013747766
[16,] -0.01058020  0.0103613874 -0.0096153675 -0.0061610132 -0.0058170446
[17,] -0.02389308  0.0048337493  0.0080844962 -0.0019111647 -0.0003003382
              [,6]          [,7]          [,8]          [,9]         [,10]
 [1,]  0.0014595059 -0.0012296670  0.0006898229  2.850487e-04  2.427770e-04
 [2,] -0.0006369214  0.0017228310  0.0014591718 -2.149749e-04 -1.504199e-03
 [3,]  0.0033216279 -0.0011246354 -0.0010373637 -9.307051e-04 -1.139473e-03
 [4,] -0.0047750460 -0.0030825755  0.0001867128 -4.225201e-03 -2.487895e-04
 [5,]  0.0005598015 -0.0014908445  0.0019128001  3.884295e-03 -1.636209e-03
 [6,] -0.0011347304  0.0033452059 -0.0037458660  1.860320e-03  2.943987e-03
 [7,]  0.0024302014  0.0015951541 -0.0006925508  5.811493e-04 -8.069821e-04
 [8,] -0.0029643835  0.0003651554 -0.0023751273 -9.954471e-05 -4.875124e-04
 [9,]  0.0007465541  0.0011717179  0.0019275043 -1.464090e-03  3.420478e-03
[10,] -0.0059994366 -0.0016752351  0.0012996016  3.629895e-03  6.301211e-04
[11,] -0.0036327702  0.0038129599  0.0028809551 -1.344179e-03 -5.797022e-04
[12,]  0.0025855698  0.0027560406 -0.0028607406 -8.567396e-04 -9.821331e-04
[13,] -0.0012901385 -0.0010433524 -0.0018628258 -1.630560e-03 -6.549801e-04
[14,]  0.0012453563 -0.0052903348 -0.0028293293  5.021320e-04  7.762288e-04
[15,]  0.0019196651  0.0001139927  0.0024349633 -2.505159e-04 -6.888229e-05
[16,]  0.0048538192 -0.0015018269  0.0031411786 -4.168970e-04  1.547531e-03
[17,]  0.0013113256  0.0015554141 -0.0005289070  6.905676e-04 -1.452261e-03
             [,11]         [,12]         [,13]         [,14]         [,15]
 [1,]  1.735903e-03  3.357651e-04 -0.0007819809 -4.689332e-04  2.424962e-04
 [2,] -2.697007e-03 -8.308690e-04  0.0008702535 -3.633743e-04  6.541015e-04
 [3,]  3.484747e-04  1.146295e-03 -0.0010299786 -2.508429e-03 -3.093835e-04
 [4,] -6.830433e-04  2.605353e-04  0.0005183371  4.800939e-04 -6.660575e-04
 [5,]  1.780341e-03 -3.679032e-05  0.0009194115  1.338875e-03 -8.293689e-04
 [6,]  5.973712e-05  5.684976e-04  0.0006798086 -4.290056e-04 -6.350696e-04
 [7,] -6.884953e-04 -1.157906e-04  0.0002139087  1.407095e-04  2.353977e-04
 [8,]  9.495744e-04 -4.948673e-04 -0.0023281668  1.090651e-03  1.673164e-03
 [9,] -2.343872e-05 -6.581122e-04 -0.0007433127  5.849794e-04 -5.206192e-04
[10,] -1.616418e-03  6.113571e-04 -0.0003307898 -8.950768e-04  1.773686e-04
[11,]  2.441005e-03 -4.108357e-04  0.0011290745 -1.097494e-03  3.902473e-04
[12,] -3.873196e-04  8.073065e-04  0.0013386978  8.085524e-04  6.664049e-04
[13,]  6.843642e-04 -4.443817e-05  0.0001653388  4.669849e-04 -1.043686e-03
[14,]  1.857194e-04 -8.394541e-04  0.0013142186 -2.382055e-04  6.112275e-04
[15,] -8.277681e-04  3.095341e-03 -0.0006625153  1.068759e-03 -6.823171e-06
[16,] -3.140994e-04 -1.290209e-03  0.0002799259 -6.091822e-05  8.260400e-04
[17,] -9.475289e-04 -2.103732e-03 -0.0015522308  8.183156e-05 -1.465441e-03
             [,16]
```

```
 [1,] -1.705869e-04
 [2,]  7.160412e-04
 [3,] -2.203327e-05
 [4,] -6.396758e-04
 [5,] -1.374811e-04
 [6,]  1.342708e-04
 [7,]  5.459053e-05
 [8,]  1.243076e-04
 [9,] -1.806350e-05
[10,] -3.432585e-04
[11,] -3.609494e-05
[12,] -7.799437e-04
[13,]  1.185534e-03
[14,]  3.246770e-05
[15,]  2.836872e-04
[16,]  5.521693e-05
[17,] -4.389782e-04
```

ML

Apply clustering (K-means, Hierarchical Clustering) and classification (Support Vector Machine, Random Forest,k-Nearest Neighbors) methods to:(1) Persistence landscapes (2) Functional principal component scores obtained from the persistence landscapes.

```
# ================================================================
# 0. Environment Setup, Helper Functions & Data Preparation
# ================================================================

# 0.1 Load Libraries
suppressPackageStartupMessages({
  library(e1071)
  library(randomForest)
  library(class)
  library(cluster)
  library(mclust)
  library(ggplot2)
  library(dplyr)
  library(tidyr)
  library(patchwork)
  library(gridExtra)
  library(grid)
  library(pROC)
```

```r
  library(RColorBrewer)
  library(ggdendro)
  library(Rtsne)
  library(knitr)
  library(dendextend)
})

# 0.2 Global Parameters
PARAM <- list(
  seed = 123,
  k_range = 2:6,          # for clustering
  rf_ntree = 500,
  knn_k_range = 1:15      # for KNN tuning
)
set.seed(PARAM$seed)

# 0.3 LaTeX replacement
sanitize_latex <- function(x) {
  if (is.character(x)) {
    x <- gsub("\\", "\\textbackslash{}", x)
    x <- gsub("\{", "\\{", x)
    x <- gsub("\}", "\\}", x)
    x <- gsub("_", "\\_", x)
    x <- gsub("%", "\\%", x)
    x <- gsub("#", "\\#", x)
    x <- gsub("&", "\\&", x)
    x <- gsub("\$", "\\$", x)
  }
  return(x)
}

# 0.4 Helper Functions
calc_metrics <- function(pred, true) {
  stopifnot(length(pred) == length(true))
  pred_fac <- factor(pred, levels = c(0, 1))
  true_fac <- factor(true, levels = c(0, 1))
  cm <- table(Pred = pred_fac, True = true_fac)
  TN <- cm[1,1]; FP <- cm[2,1]
  FN <- cm[1,2]; TP <- cm[2,2]
  total <- sum(cm)
  acc <- (TP + TN) / total
  sens <- ifelse(TP + FN > 0, TP / (TP + FN), NA)
```

```
  spec <- ifelse(TN + FP > 0, TN / (TN + FP), NA)
  prec <- ifelse(TP + FP > 0, TP / (TP + FP), NA)
  f1 <- ifelse(!is.na(prec) & !is.na(sens) & (prec + sens) > 0,
               2 * prec * sens / (prec + sens), NA)
  bal_acc <- (sens + spec) / 2
  denom <- sqrt(as.numeric(TP + FP) * (TP + FN) * (TN + FP) * (TN + FN))
  mcc <- ifelse(denom > 0, (TP * TN - FP * FN) / denom, NA)
  return(c(Accuracy = acc, Balanced_Accuracy = bal_acc,
           Sensitivity = sens, Specificity = spec,
           Precision = prec, F1 = f1, MCC = mcc))
}

extract_prob1 <- function(prob_mat) {
  if (is.null(prob_mat)) return(0)
  if (is.vector(prob_mat)) {
    if (!is.null(names(prob_mat)) && "1" %in% names(prob_mat))
      return(as.numeric(prob_mat["1"]))
    if (length(prob_mat) == 2) return(as.numeric(prob_mat[2]))
    return(as.numeric(prob_mat[1]))
  }
  cols <- colnames(prob_mat)
  if (!is.null(cols) && "1" %in% cols)
    return(as.numeric(prob_mat[, "1"]))
  else if (!is.null(cols) && all(cols == "0"))
    return(rep(0, nrow(prob_mat)))
  else
    return(rep(1, nrow(prob_mat)))
}

scale_train_test <- function(train_x, test_x) {
  means <- colMeans(train_x)
  sds <- apply(train_x, 2, sd)
  sds[sds == 0] <- 1
  return(list(train = scale(train_x, center = means, scale = sds),
              test = scale(test_x, center = means, scale = sds)))
}

# ---- Clustering evaluation ----
evaluate_clustering <- function(data, labels, k_range,
                                method = "kmeans",
                                dist_mat = NULL,
                                linkage = "ward.D2") {
```

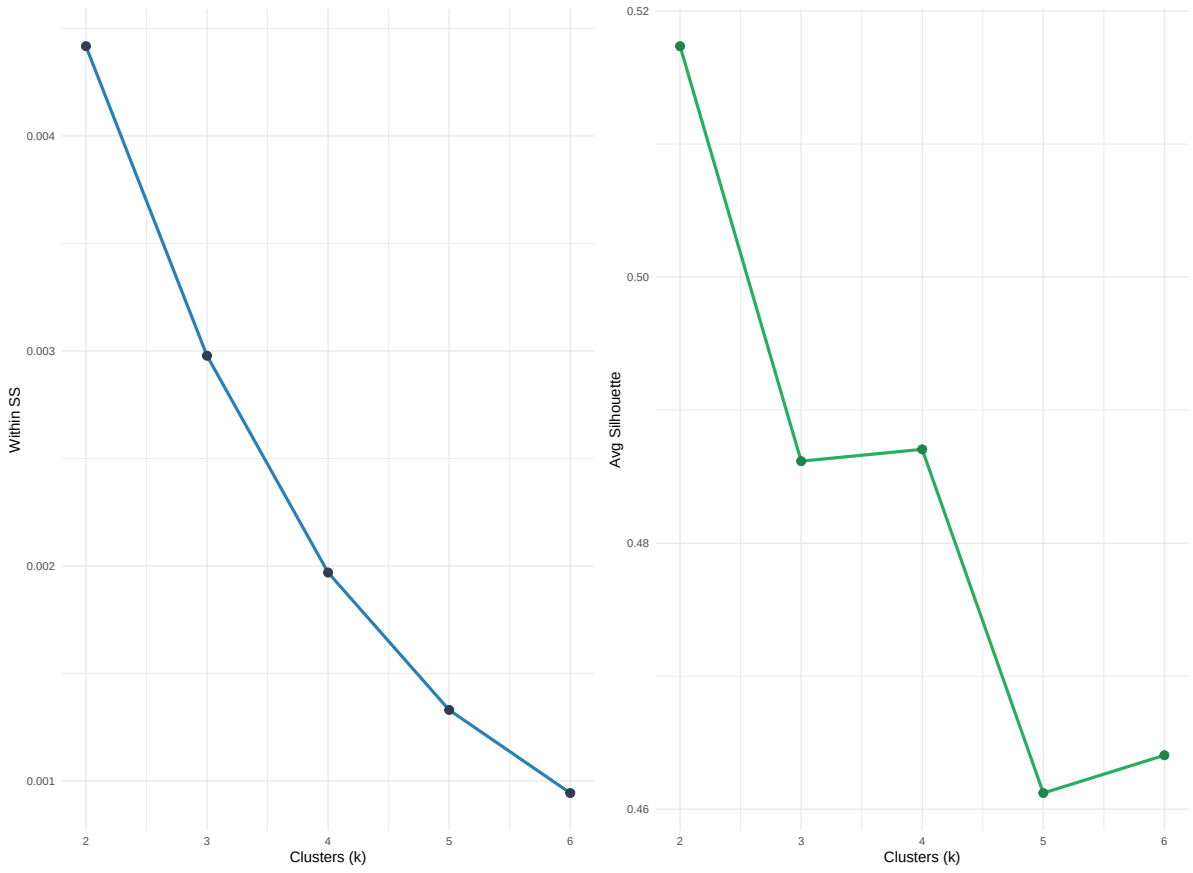
```
  if (is.null(dist_mat)) dist_mat <- dist(data)
  if (method == "kmeans") {
    sil <- sapply(k_range, function(k) {
      km <- kmeans(data, centers = k, nstart = 25)
      mean(silhouette(km$cluster, dist_mat)[, 3])
    })
    best_k <- k_range[which.max(sil)]
    km_final <- kmeans(data, centers = best_k, nstart = 25)
    ari <- adjustedRandIndex(km_final$cluster, labels)
    clusters <- km_final$cluster
  } else if (method == "hierarchical") {
    hc <- hclust(dist_mat, method = linkage)
    sil <- sapply(k_range, function(k) {
      mean(silhouette(cutree(hc, k = k), dist_mat)[, 3])
    })
    best_k <- k_range[which.max(sil)]
    clusters <- cutree(hc, k = best_k)
    ari <- adjustedRandIndex(clusters, labels)
  }
  return(list(best_k = best_k, max_sil = max(sil), ari = ari, clusters = clusters))
}

get_palette <- function(n) {
  if (n <= 8) brewer.pal(max(3, n), "Set1")[1:n] else
    colorRampPalette(brewer.pal(8, "Set1"))(n)
}

plot_dendro_gg <- function(dist_mat, method_name, title_str, m_labels) {
  hc <- hclust(dist_mat, method = method_name)
  ddata <- dendro_data(hc, type = "rectangle")
  leaf_order <- order.dendrogram(as.dendrogram(hc))
  raw_labels <- m_labels[leaf_order]
  leaf_data <- ddata$labels
  leaf_data$label_raw <- raw_labels
  leaf_data$Group <- ifelse(grepl("^A", raw_labels), "Asymptomatic", "Symptomatic")
  leaf_data$color <- ifelse(leaf_data$Group == "Asymptomatic", "#2C3E50", "#E74C3C")
  p <- ggplot() +
    geom_segment(data = segment(ddata), aes(x = x, y = y, xend = xend, yend = yend),
                 color = "#34495E", linewidth = 0.4) +
    geom_text(data = leaf_data,
              aes(x = x, y = -max(ddata$segments$y) * 0.05,
                  label = label_raw, color = color),
```

```
              angle = 90, hjust = 1, vjust = 0.5, size = 2.6, fontface = "bold") +
    scale_color_identity() +
    scale_y_continuous(expand = expansion(mult = c(0.38, 0.08))) +
    labs(title = title_str, y = "Height") +
    theme_minimal(base_size = 7.5) +
    theme(plot.title = element_text(size = 8, face = "bold", hjust = 0.5),
          axis.title.x = element_blank(),
          axis.text = element_blank(),
          axis.ticks = element_blank(),
          panel.grid = element_blank(),
          legend.position = "none",
          plot.margin = ggplot2::margin(2, 2, 8, 2, "mm"))
  return(p)
}

# ---- SVM decision boundary visualization (based on PC1/PC2) ----
plot_svm_boundaries <- function(pca_data, labels) {
  df <- data.frame(
    PC1 = pca_data[, 1], PC2 = pca_data[, 2],
    Group = factor(ifelse(labels == 1, "Symptomatic", "Asymptomatic"),
                   levels = c("Asymptomatic", "Symptomatic"))
  )

  pc1_seq <- seq(min(df$PC1) - 0.01, max(df$PC1) + 0.01, length.out = 80)
  pc2_seq <- seq(min(df$PC2) - 0.01, max(df$PC2) + 0.01, length.out = 80)
  grid <- expand.grid(PC1 = pc1_seq, PC2 = pc2_seq)

  kernels <- c("linear", "radial", "polynomial")
  titles <- c("Linear Kernel", "Radial (RBF) Kernel", "Polynomial Kernel")
  plot_list <- list()

  for (i in 1:3) {
    m_svm <- tryCatch({
      svm(Group ~ PC1 + PC2, data = df,
          kernel = kernels[i], cost = 1, gamma = 0.1, scale = TRUE)
    }, error = function(e) NULL)

    if (is.null(m_svm)) {
      p <- ggplot() + annotate("text", x = 0,
                               y = 0, label = paste(titles[i], "\n(Fit Failed)"),
                               size = 3) +
        theme_void() + labs(title = titles[i])
```

```
      plot_list[[i]] <- p
      next
    }

    grid$Pred <- predict(m_svm, grid)

    p <- ggplot() +
      geom_tile(data = grid, aes(x = PC1, y = PC2, fill = Pred), alpha = 0.2) +
      geom_point(data = df, aes(x = PC1, y = PC2, color = Group), size = 2, alpha = 0.9) +
      scale_fill_manual(values = c("Asymptomatic" = "#2C3E50",
                                   "Symptomatic" = "#2980B9")) +
      scale_color_manual(values = c("Asymptomatic" = "#14252F",
                                    "Symptomatic" = "#154360")) +
      labs(title = sanitize_latex(titles[i]), x = "PC1 Score", y = "PC2 Score") +
      theme_minimal(base_size = 8) +
      theme(
        plot.title = element_text(size = 8.5, face = "bold", hjust = 0.5),
        legend.position = "none",
        plot.margin = ggplot2::margin(2, 2, 2, 2, "mm")
      )
    plot_list[[i]] <- p
  }

  wrap_plots(plot_list, ncol = 3) +
    plot_annotation(title = "", #"SVM Decision Boundaries across Kernels (PC1 vs PC2)"
                    theme = theme(plot.title = element_text(size = 10, face = "bold")))
}

# 0.5 Data Extraction & Preparation
pca_scores <- pca$scores[, 1:n_pc_keep, drop = FALSE]
if (!exists("m")) {
  m <- c("A1", "A2", "A3", "A4", "A5", "A6", "A7", "A8",
         "S1", "S2", "S3", "S4", "S5", "S6", "S7", "S8", "S9")
}
rownames(DataBoth) <- m
rownames(pca_scores) <- m
group <- ifelse(grepl("^A", m), "Asymptomatic", "Symptomatic")
y_true <- as.numeric(group == "Symptomatic")

pca_df <- data.frame(
  PC1 = pca_scores[, 1], PC2 = pca_scores[, 2], ID = m,
  Group = factor(group, levels = c("Asymptomatic", "Symptomatic"))
```

```
)

dist_raw <- dist(DataBoth)
dist_pca <- dist(pca_scores)

# Unified coordinate system for plotting
if (n_pc_keep > 2) {
  set.seed(123)
  tsne_proj <- Rtsne(pca_scores, dims = 2,
                     perplexity = min(5, nrow(pca_scores) - 1),
                     check_duplicates = FALSE, pca = FALSE)
  proj_coords <- data.frame(X = tsne_proj$Y[, 1], Y = tsne_proj$Y[, 2])
  proj_label <- "t-SNE (FPCA)"
} else {
  proj_coords <- data.frame(X = pca_scores[, 1], Y = pca_scores[, 2])
  proj_label <- "PC1/PC2"
}

# ================================================================
# 1. Unsupervised Clustering Analysis
# ================================================================

# Step 1.1: Parameter Diagnostics (WSS and Silhouette)
wss_vals <- sapply(PARAM$k_range,
                   function(k) kmeans(pca_scores, centers = k, nstart = 25)$tot.withinss)
sil_vals <- sapply(PARAM$k_range,
                   function(k)
                     mean(silhouette(kmeans(pca_scores,
                                            centers = k,
                                            nstart = 25)$cluster, dist_pca)[, 3]))

df_diag <- data.frame(k = PARAM$k_range, WSS = wss_vals, Silhouette = sil_vals)

p_wss <- ggplot(df_diag, aes(x = k, y = WSS)) +
  geom_line(color = "#2980B9", linewidth = 0.8) +
  geom_point(color = "#2C3E50", size = 2) +
  labs(title = " ", x = "Clusters (k)", y = "Within SS") + #"Elbow Method (WSS)"
  theme_minimal(base_size = 8) + theme(plot.title = element_text(face = "bold"))

p_sil <- ggplot(df_diag, aes(x = k, y = Silhouette)) +
  geom_line(color = "#27AE60", linewidth = 0.8) +
  geom_point(color = "#1E8449", size = 2) +
```

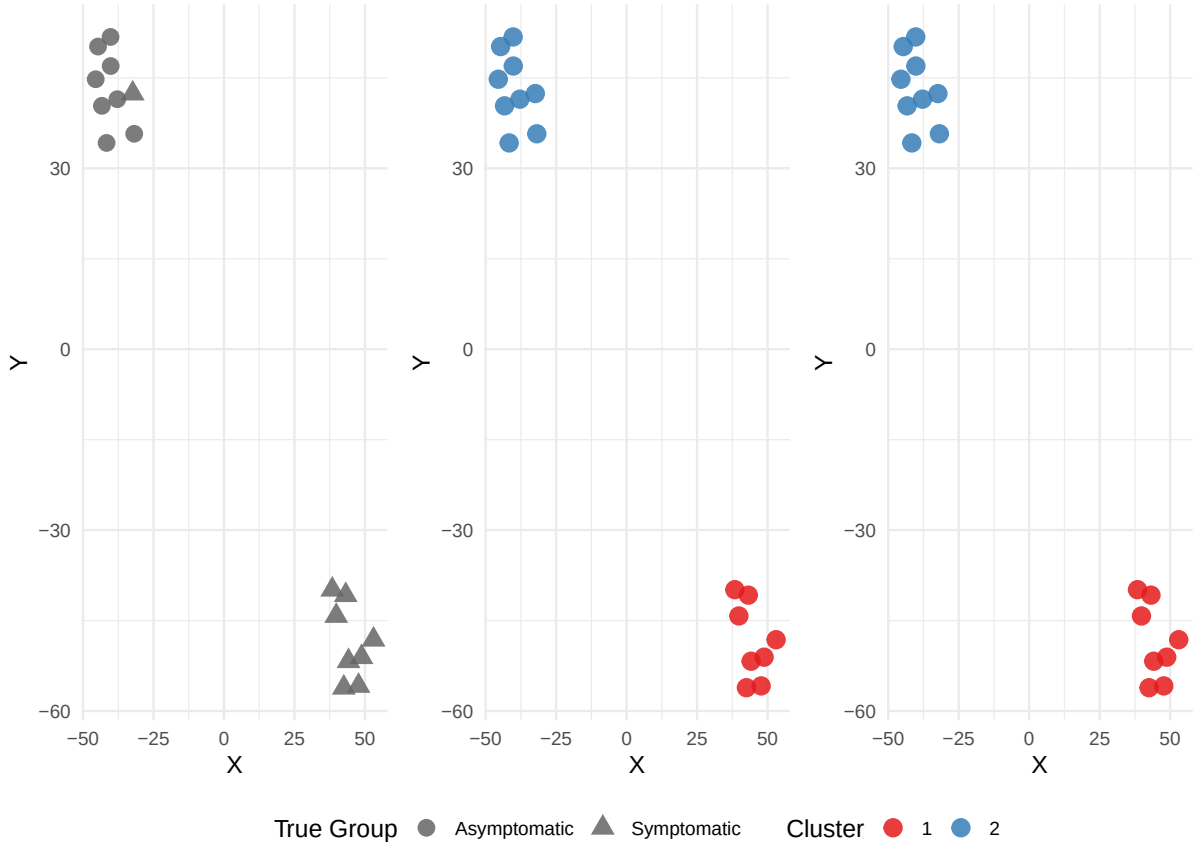
```
  labs(title = " ", x = "Clusters (k)", y = "Avg Silhouette") + # "Silhouette Analysis"
  theme_minimal(base_size = 8) + theme(plot.title = element_text(face = "bold"))

diag_grid <- wrap_plots(p_wss, p_sil, ncol = 2) +
  plot_annotation(title = " ", #Section 1.1: Clustering Parameter Diagnostics
                  theme = theme(plot.title = element_text(size = 10, face = "bold")))
print(diag_grid)
```




```
# ---- Ground Truth ----
p_gt_cluster <- ggplot(data.frame(X = proj_coords$X, Y = proj_coords$Y,
                                  Group = factor(ifelse(y_true == 1, "Symptomatic", "Asymptomatic"))),
                       aes(x = X, y = Y, shape = Group)) +
  geom_point(size = 4, alpha = 0.85, color = "gray40") +
  scale_shape_manual(values = c(16, 17)) +
  labs(title = " ", shape = "True Group") + #"Ground Truth", , subtitle = proj_label
  theme_minimal(base_size = 13) + theme(legend.position = "bottom")
```

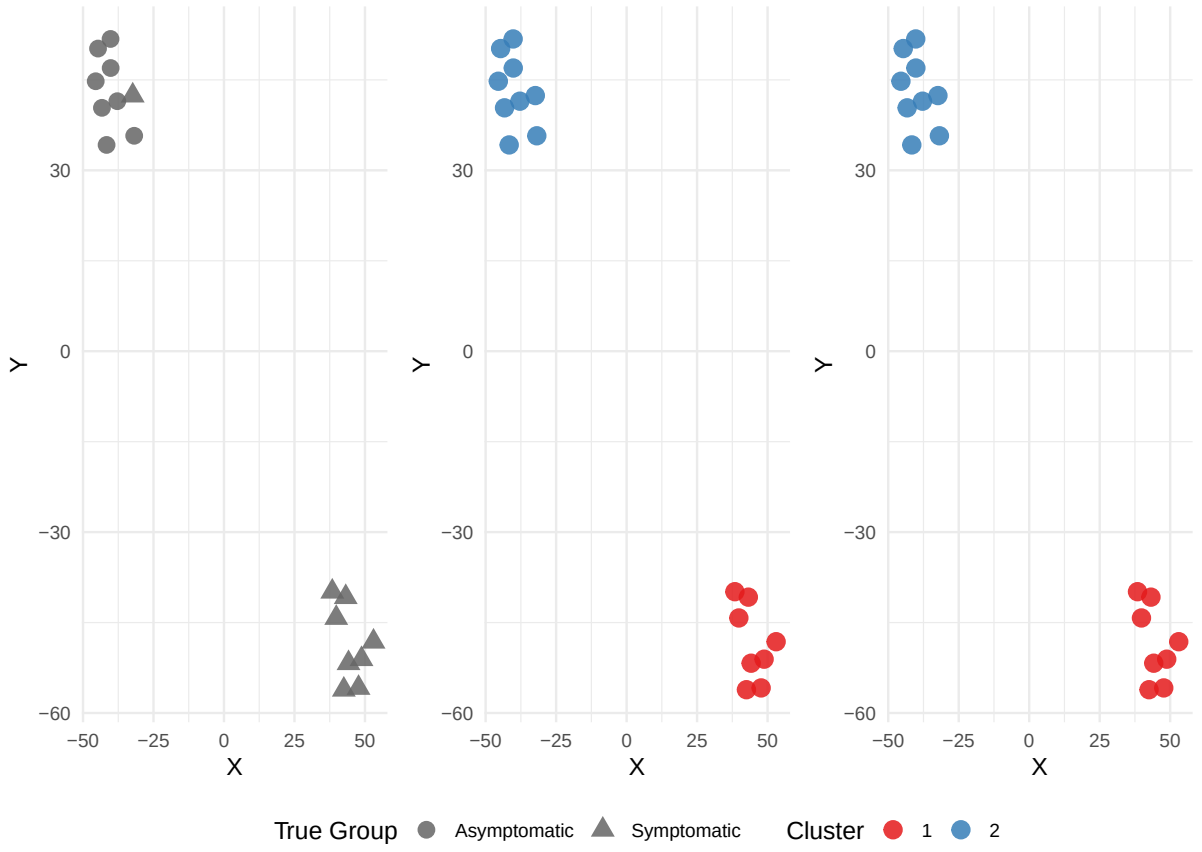
```r
# Step 1.2: K-Means Clustering
km_raw <- evaluate_clustering(DataBoth, y_true, PARAM$k_range, "kmeans", dist_raw)
km_pca <- evaluate_clustering(pca_scores, y_true, PARAM$k_range, "kmeans", dist_pca)

pca_df$Cluster_KM_Raw <- factor(km_raw$clusters)
pca_df$Cluster_KM_PCA <- factor(km_pca$clusters)

all_clusters_km <- unique(c(km_raw$clusters, km_pca$clusters))
palette_km <- get_palette(length(all_clusters_km))
names(palette_km) <- sort(all_clusters_km)

km_raw_df <- data.frame(X = proj_coords$X, Y = proj_coords$Y,
                        Cluster = factor(km_raw$clusters))
p_km_raw <- ggplot(km_raw_df, aes(x = X, y = Y, color = Cluster)) +
  geom_point(size = 4, alpha = 0.85) +
  scale_color_brewer(palette = "Set1") +
  labs(title = " ", #K-means (Raw Data)
       #subtitle = paste0("k = ", km_raw$best_k, ", based on raw landscapes (", proj_label, "
       color = "Cluster") +
  theme_minimal(base_size = 13) + theme(legend.position = "bottom")

km_pca_df <- data.frame(X = proj_coords$X, Y = proj_coords$Y,
                        Cluster = factor(km_pca$clusters))
p_km_pca <- ggplot(km_pca_df, aes(x = X, y = Y, color = Cluster)) +
  geom_point(size = 4, alpha = 0.85) +
  scale_color_brewer(palette = "Set1") +
  labs(title = " ", # "K-means (FPCA Scores)"
       # subtitle = paste0("k = ", km_pca$best_k, ", based on ", n_pc_keep, " PCs (", proj_la
       color = "Cluster") +
  theme_minimal(base_size = 13) + theme(legend.position = "bottom")

km_scatter_panel <- (wrap_plots(p_gt_cluster, p_km_raw, p_km_pca, ncol = 3, guides = "collect
                      theme(legend.position = "bottom") &
                      plot_annotation(title = "Section 1.2: K-Means Clustering Results Acros
                      theme(plot.title = element_text(size = 10, face = "bold")))
print(km_scatter_panel)
```

Section 1.2: K−Means Clustering Results Across Data Inputs

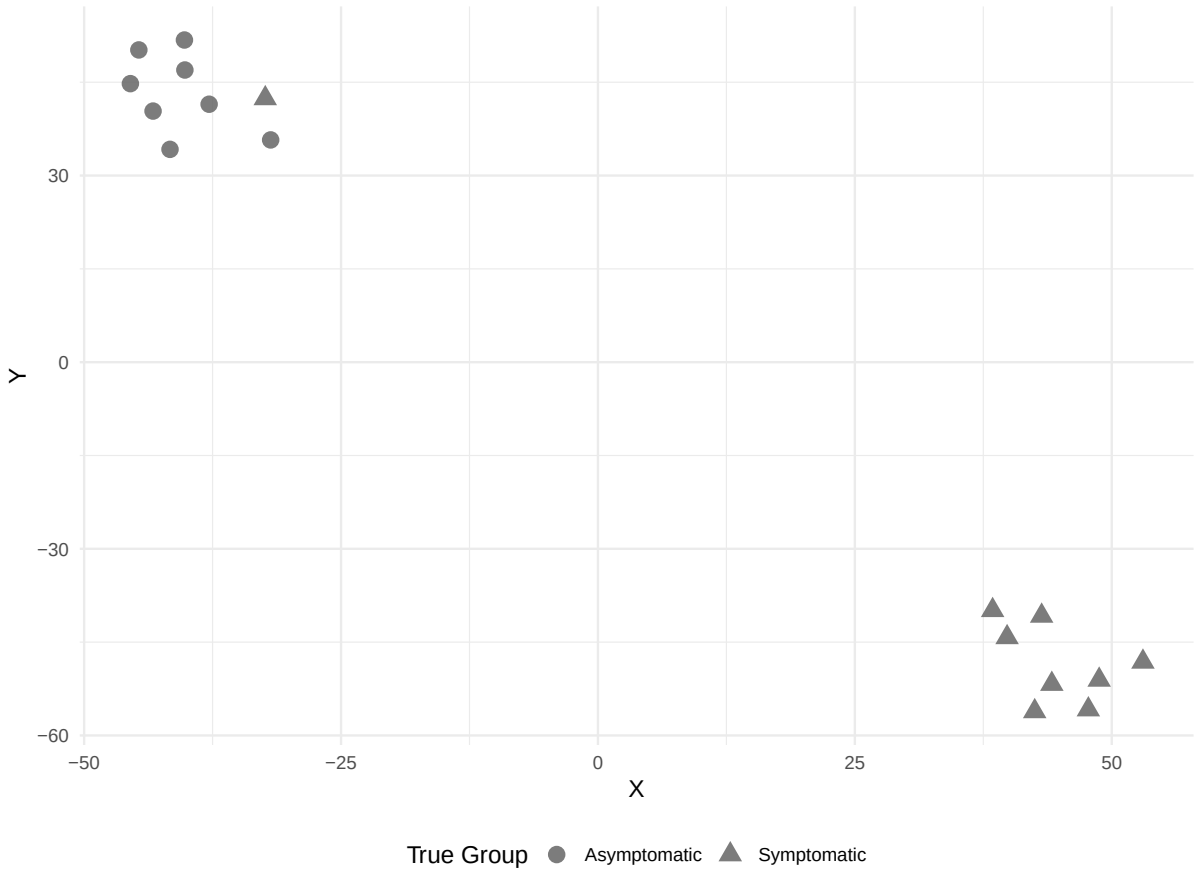



```
print(km_scatter_panel)
```

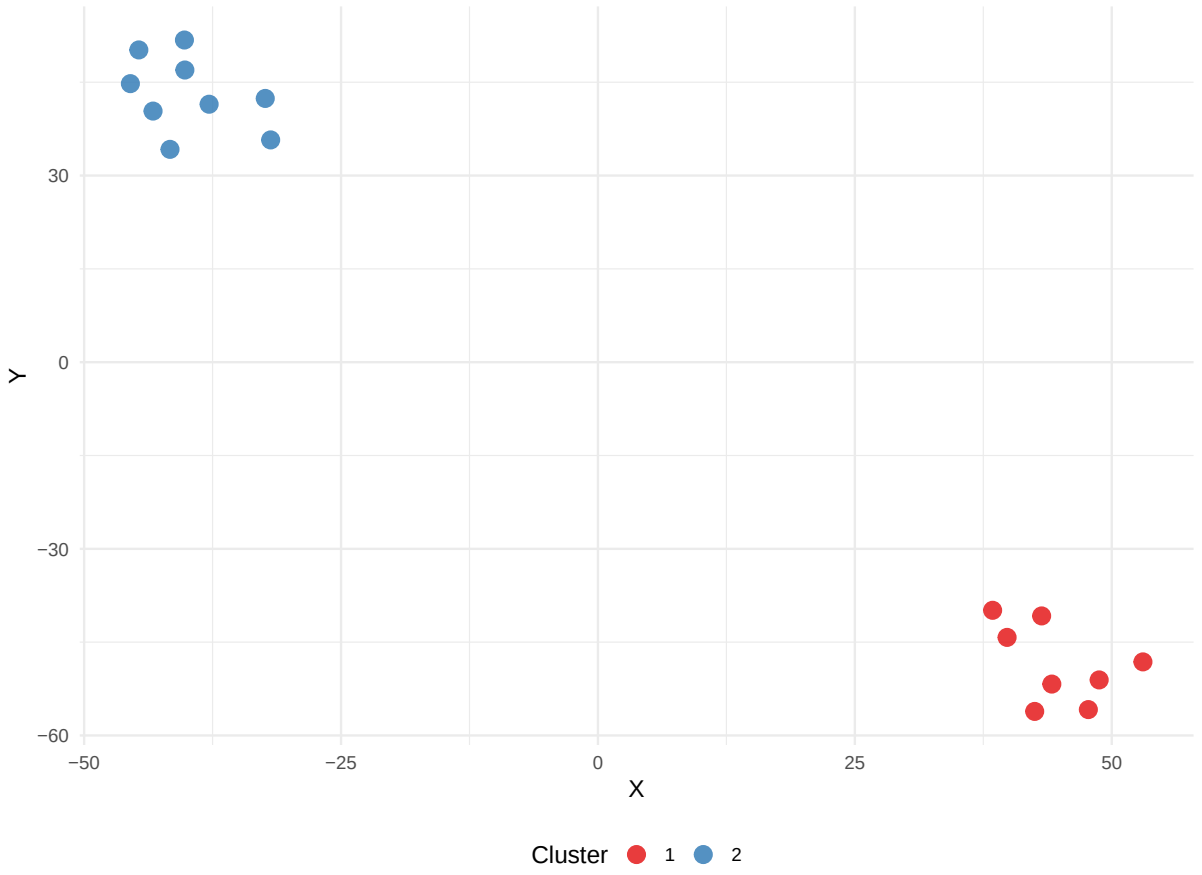
Section 1.2: K−Means Clustering Results Across Data Inputs



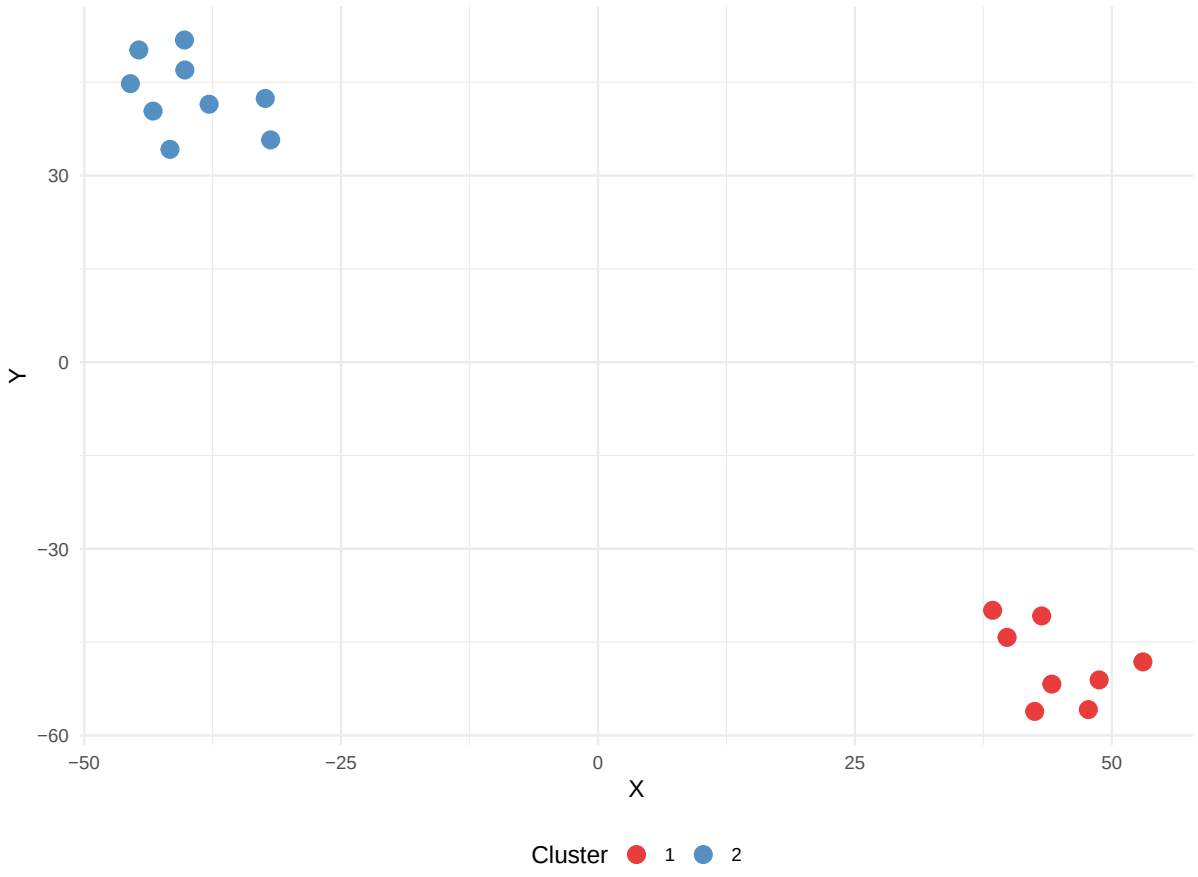

```
print(p_gt_cluster) #
```




```
print(p_km_raw) #
```




```
print(p_km_pca) #
```




```
p_km_raw     <- p_km_raw     + theme(aspect.ratio = 1.8)
p_km_pca     <- p_km_pca     + theme(aspect.ratio = 1.8)


# Additional: K-Means with k = 3 (manual inspection)
set.seed(PARAM$seed)
km_raw_k3 <- kmeans(DataBoth, centers = 3, nstart = 25)
km_pca_k3 <- kmeans(pca_scores, centers = 3, nstart = 25)

sil_raw_k3 <- mean(silhouette(km_raw_k3$cluster, dist_raw)[, 3])
sil_pca_k3 <- mean(silhouette(km_pca_k3$cluster, dist_pca)[, 3])
ari_raw_k3 <- adjustedRandIndex(km_raw_k3$cluster, y_true)
ari_pca_k3 <- adjustedRandIndex(km_pca_k3$cluster, y_true)

cat("\n--- K-means k=3 (Manual Inspection) ---\n")
```

```
--- K-means k=3 (Manual Inspection) ---
```

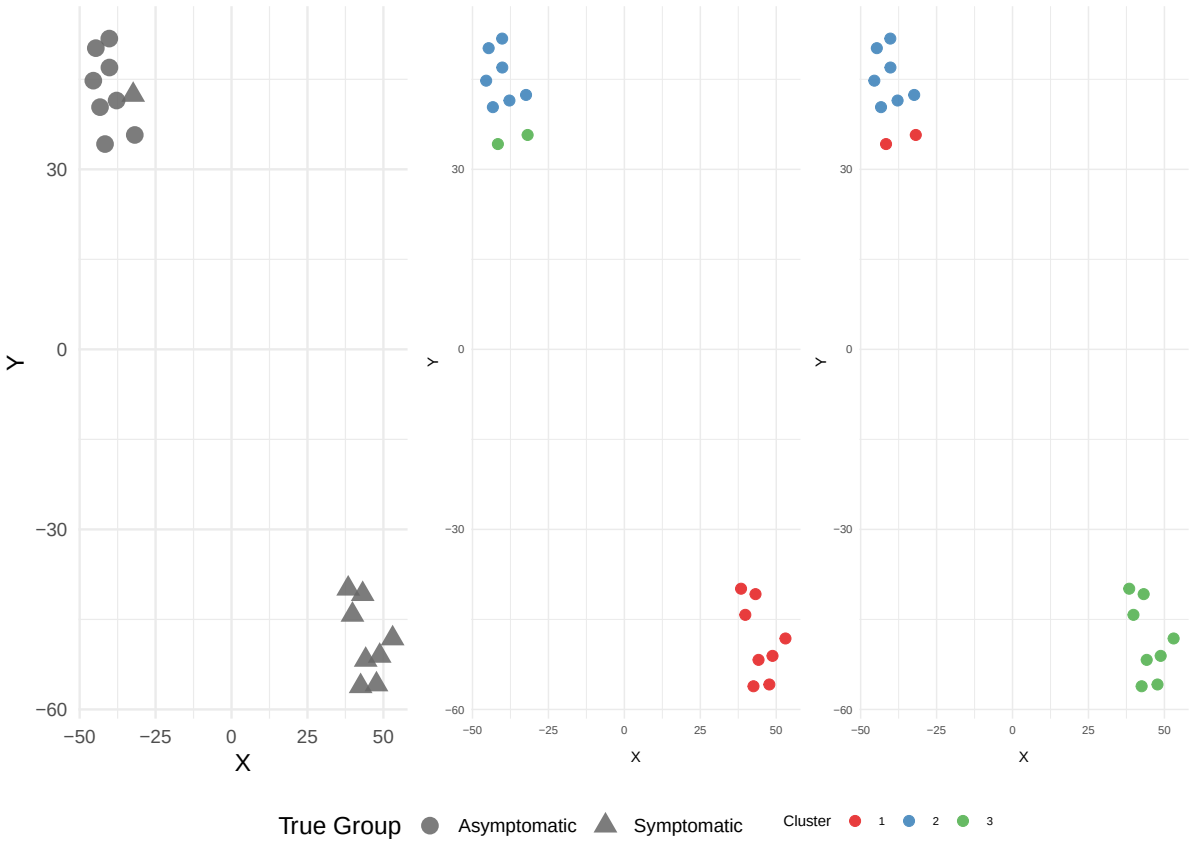
```
cat(sprintf("Raw data:   Silhouette = %.3f, ARI = %.3f\n", sil_raw_k3, ari_raw_k3))
```

```
Raw data:   Silhouette = 0.444, ARI = 0.612
```

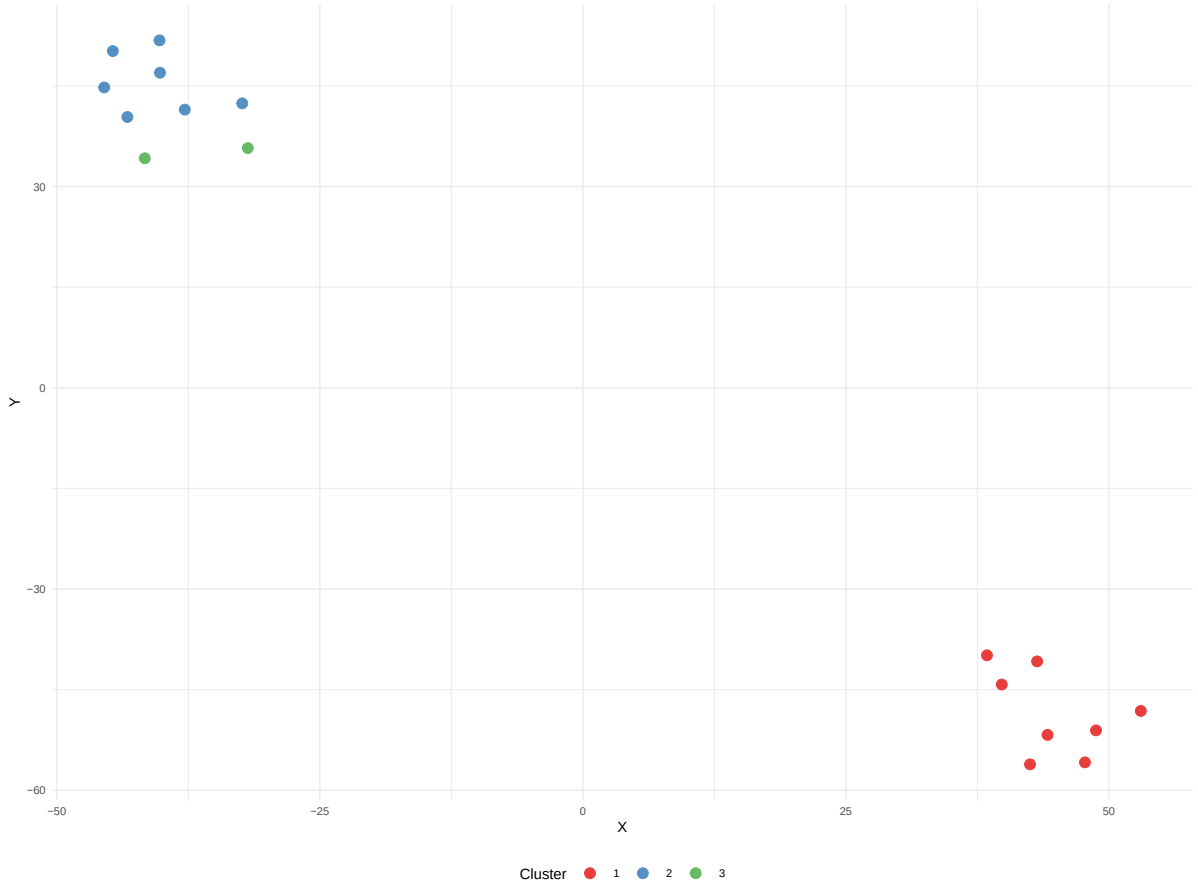
```
cat(sprintf("FPCA scores: Silhouette = %.3f, ARI = %.3f\n", sil_pca_k3, ari_pca_k3))
```

```
FPCA scores: Silhouette = 0.486, ARI = 0.612
```

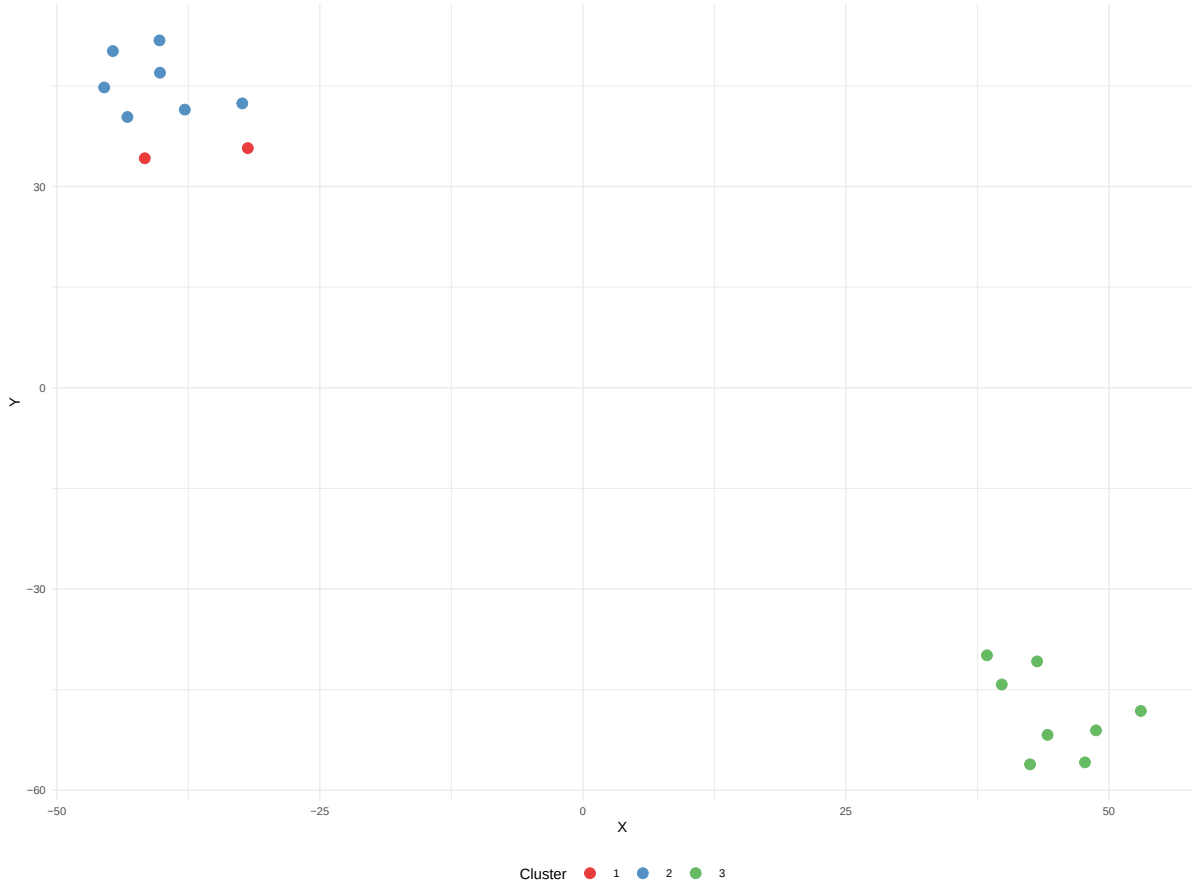
```
df_raw_k3 <- data.frame(X = proj_coords$X, Y = proj_coords$Y,
                        Cluster = factor(km_raw_k3$cluster))
p_km_raw_k3 <- ggplot(df_raw_k3, aes(x = X, y = Y, color = Cluster)) +
  geom_point(size = 2.5, alpha = 0.85) +
  scale_color_brewer(palette = "Set1") +
  labs(title = " ", #K-means (Raw, k=3)
#       subtitle = paste0("Silhouette = ", round(sil_raw_k3, 3),
#                         ", ARI = ", round(ari_raw_k3, 3),
 #                        " (", proj_label, ")"),
       color = "Cluster") +
  theme_minimal(base_size = 8) + theme(legend.position = "bottom")

df_pca_k3 <- data.frame(X = proj_coords$X, Y = proj_coords$Y,
                        Cluster = factor(km_pca_k3$cluster))
p_km_pca_k3 <- ggplot(df_pca_k3, aes(x = X, y = Y, color = Cluster)) +
  geom_point(size = 2.5, alpha = 0.85) +
  scale_color_brewer(palette = "Set1") +
  labs(title = " ", #K-means (FPCA, k=3)
 #      subtitle = paste0("Silhouette = ", round(sil_pca_k3, 3),
 #                        ", ARI = ", round(ari_pca_k3, 3), " (", proj_label, ")"),
       color = "Cluster") +
  theme_minimal(base_size = 8) + theme(legend.position = "bottom")

km3_scatter_panel <- (wrap_plots(p_gt_cluster, p_km_raw_k3, p_km_pca_k3,
                                 ncol = 3, guides = "collect") &
                        theme(legend.position = "bottom") &
                        plot_annotation(title = " ") &
                        #K-Means Clustering with k = 3 (Manual Inspection - Unified Coordinat
                        theme(plot.title = element_text(size = 10, face = "bold")))
print(km3_scatter_panel)
```




```
print(p_km_raw_k3) #
```

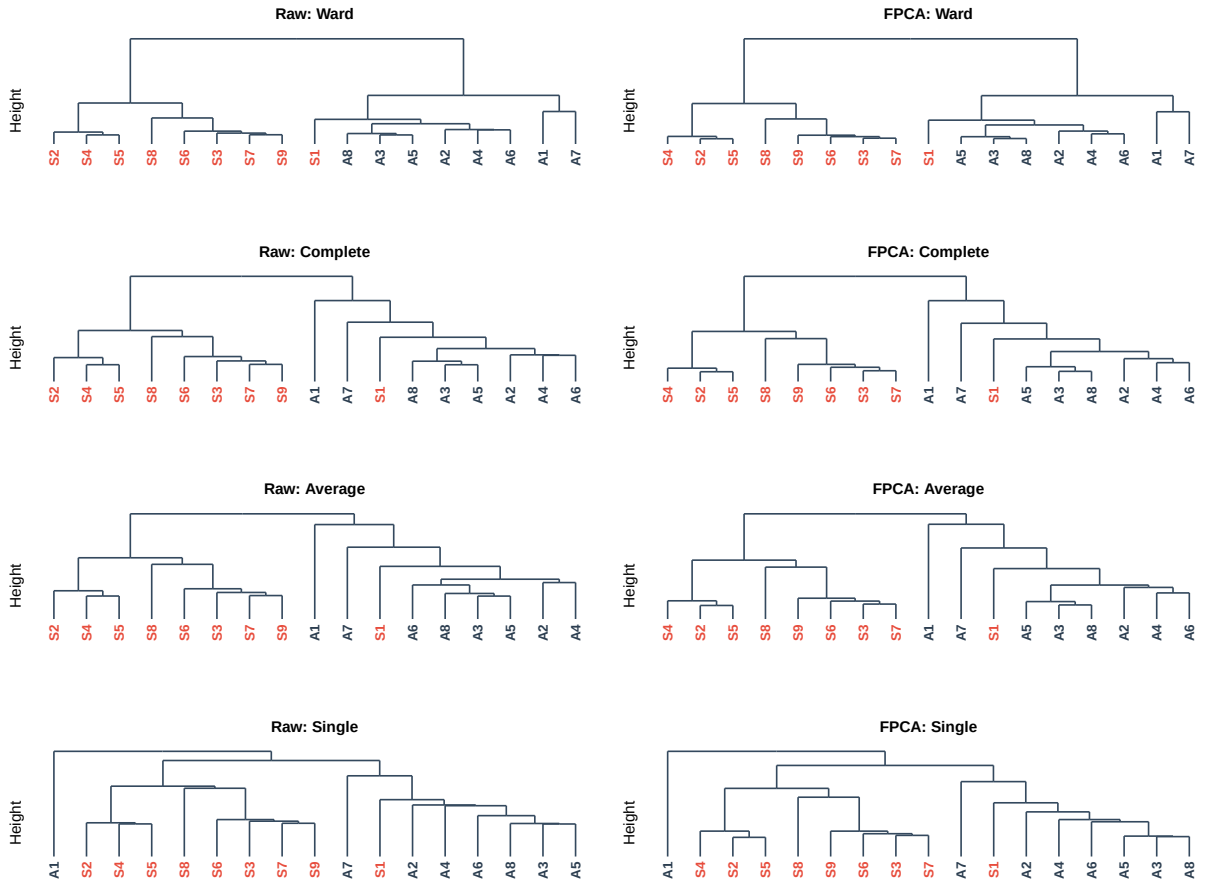



```
print(p_km_pca_k3) #
```



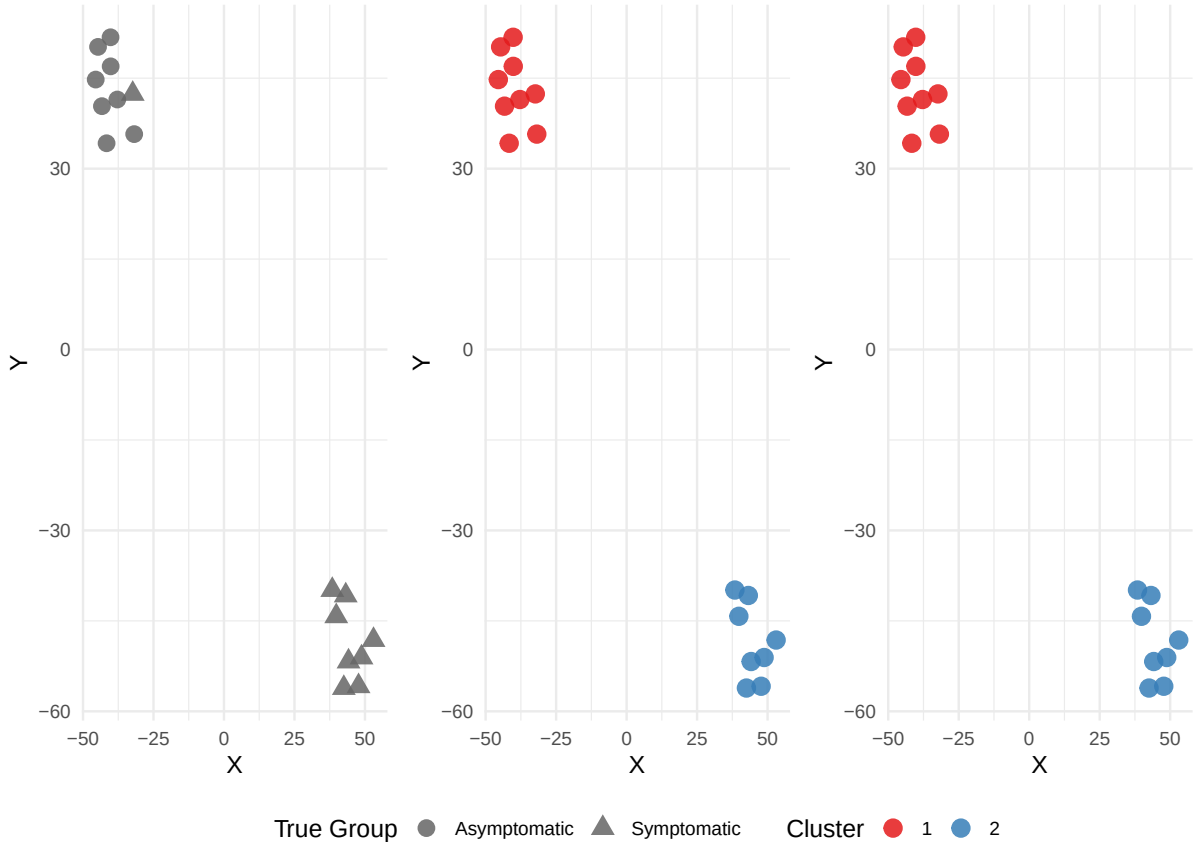

```
# Step 1.3A: Hierarchical Dendrogram Comparison (2x4 Matrix)
linkages <- c("ward.D2", "complete", "average", "single")
linkage_titles <- c("Ward", "Complete", "Average", "Single")
dend_plots <- list()
idx <- 1
for (i in 1:4) {
  dend_plots[[idx]] <- plot_dendro_gg(dist_raw, linkages[i], paste0("Raw: ", linkage_titles[
  idx <- idx + 1
  dend_plots[[idx]] <- plot_dendro_gg(dist_pca, linkages[i], paste0("FPCA: ", linkage_titles
  idx <- idx + 1
}
dendro_2x4_grid <- wrap_plots(dend_plots, ncol = 2) #+
#  plot_annotation(
#    title = "Section 1.3A: Hierarchical Linkage Structure Comparison (2x4 Matrix)",
#    subtitle = "Leaf Labels: Dark = True Asymptomatic, Red = True Symptomatic",
#    theme = theme(plot.title = element_text(size = 10, face = "bold"),
#                  plot.subtitle = element_text(size = 8, face = "italic"))
```

```
#  )
print(dendro_2x4_grid)
```

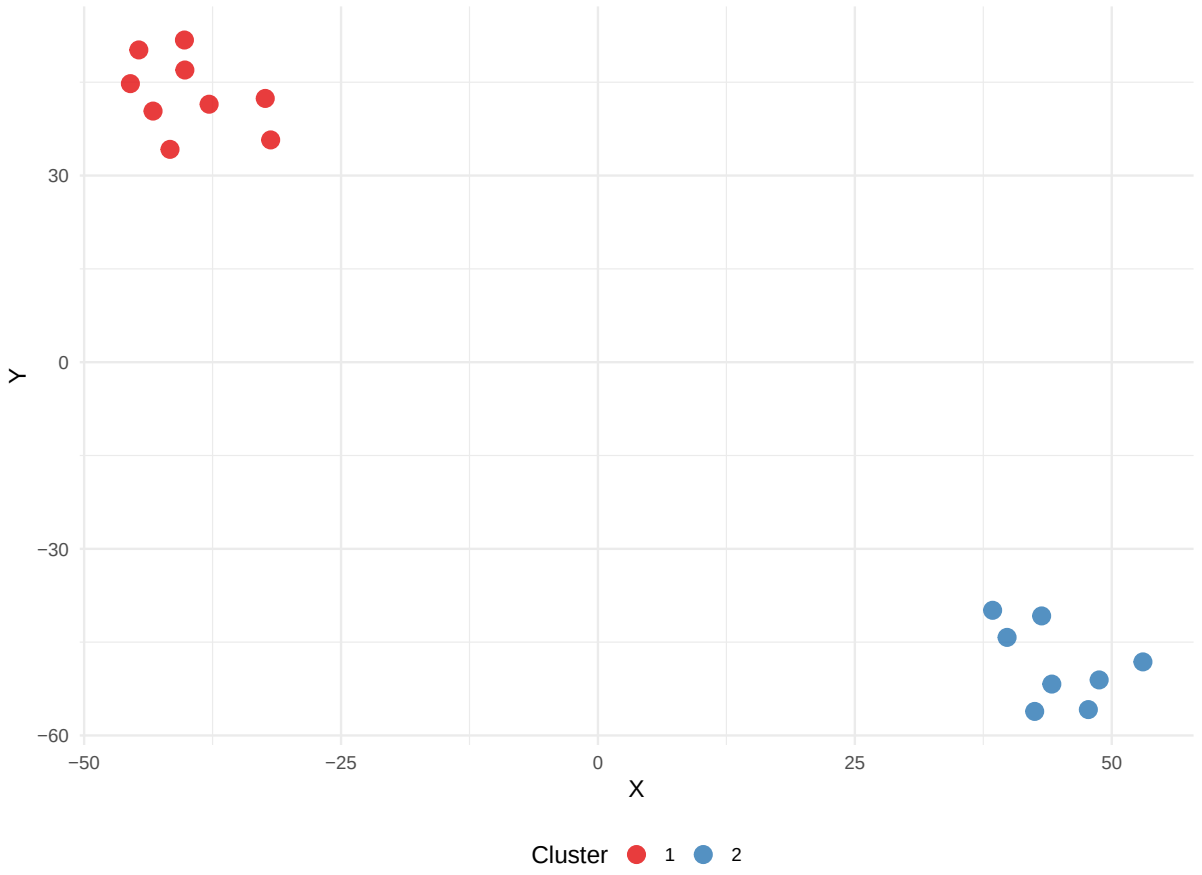



```
# Step 1.3B: Hierarchical Scatter Comparison
hc_raw <- evaluate_clustering(DataBoth, y_true, PARAM$k_range,
                              "hierarchical", dist_raw, "ward.D2")
hc_pca <- evaluate_clustering(pca_scores, y_true, PARAM$k_range,
                              "hierarchical", dist_pca, "ward.D2")

pca_df$Cluster_HC_Raw <- factor(hc_raw$clusters)
pca_df$Cluster_HC_PCA <- factor(hc_pca$clusters)

all_clusters_hc <- unique(c(hc_raw$clusters, hc_pca$clusters))
palette_hc <- get_palette(length(all_clusters_hc))
names(palette_hc) <- sort(all_clusters_hc)

hc_raw_df <- data.frame(X = proj_coords$X, Y = proj_coords$Y,
```

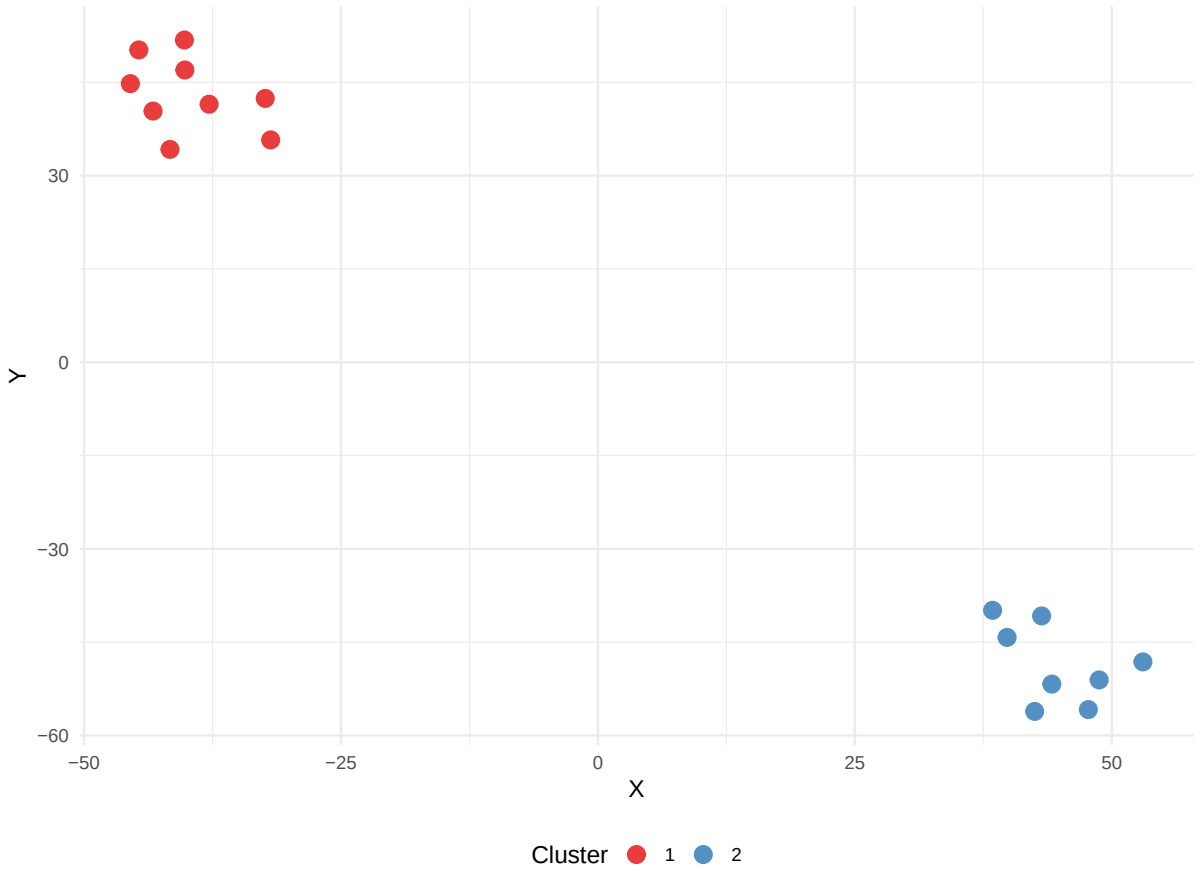
```
                         Cluster = factor(hc_raw$clusters))
p_hc_raw <- ggplot(hc_raw_df, aes(x = X, y = Y, color = Cluster)) +
  geom_point(size = 4, alpha = 0.85) +
  scale_color_brewer(palette = "Set1") +
  labs(title = " ", #Hierarchical (Raw Data)
       #subtitle = paste0("k = ", hc_raw$best_k, ", based on raw landscapes (", proj_label, "
       color = "Cluster") +
  theme_minimal(base_size = 13) + theme(legend.position = "bottom")

hc_pca_df <- data.frame(X = proj_coords$X, Y = proj_coords$Y,
                         Cluster = factor(hc_pca$clusters))
p_hc_pca <- ggplot(hc_pca_df, aes(x = X, y = Y, color = Cluster)) +
  geom_point(size = 4, alpha = 0.85) +
  scale_color_brewer(palette = "Set1") +
  labs(title = " ", #Hierarchical (FPCA Scores)
      # subtitle = paste0("k = ", hc_pca$best_k, ", based on ", n_pc_keep, " PCs (", proj_lab
       color = "Cluster") +
  theme_minimal(base_size = 13) + theme(legend.position = "bottom")

hc_scatter_panel <- (wrap_plots(p_gt_cluster, p_hc_raw, p_hc_pca, ncol = 3, guides = "collect
                        theme(legend.position = "bottom") &
                        plot_annotation(title = "") &
                        #Section 1.3B: Hierarchical (Ward.D2) Clustering Results
                        theme(plot.title = element_text(size = 10, face = "bold")))
print(hc_scatter_panel)
```




```
print(p_hc_raw)
```




```
print(p_hc_pca)
```




```
p_gt_cluster <- p_gt_cluster + theme(aspect.ratio = 1.8)
p_hc_raw     <- p_hc_raw    + theme(aspect.ratio = 1.8)
p_hc_pca     <- p_hc_pca    + theme(aspect.ratio = 1.8)

# Step 1.4: Table 1 - Unsupervised Performance Summary
cluster_summary <- data.frame(
  Method = rep(c("K-means", "Hierarchical"), each = 2),
  Data = rep(c("Raw", "FPCA"), 2),
  "Best k" = c(km_raw$best_k, km_pca$best_k, hc_raw$best_k, hc_pca$best_k),
  "Max Silhouette" = round(c(km_raw$max_sil, km_pca$max_sil, hc_raw$max_sil, hc_pca$max_sil)
  ARI = round(c(km_raw$ari, km_pca$ari, hc_raw$ari, hc_pca$ari), 3),
  check.names = FALSE
) |>
  mutate(across(where(is.character), sanitize_latex))

cluster_champion <- cluster_summary |>
  arrange(desc(ARI), desc(`Max Silhouette`)) |>
```

```
  slice(1)

knitr::kable(cluster_summary,
             # caption = "Unsupervised Clustering Performance Summary",
              format = "latex", booktabs = TRUE, escape = TRUE)
```

Method	Data	Best k	Max Silhouette	ARI
K-means	Raw	2	0.482	0.764
K-means	FPCA	2	0.517	0.764
Hierarchical	Raw	2	0.482	0.764
Hierarchical	FPCA	2	0.517	0.764

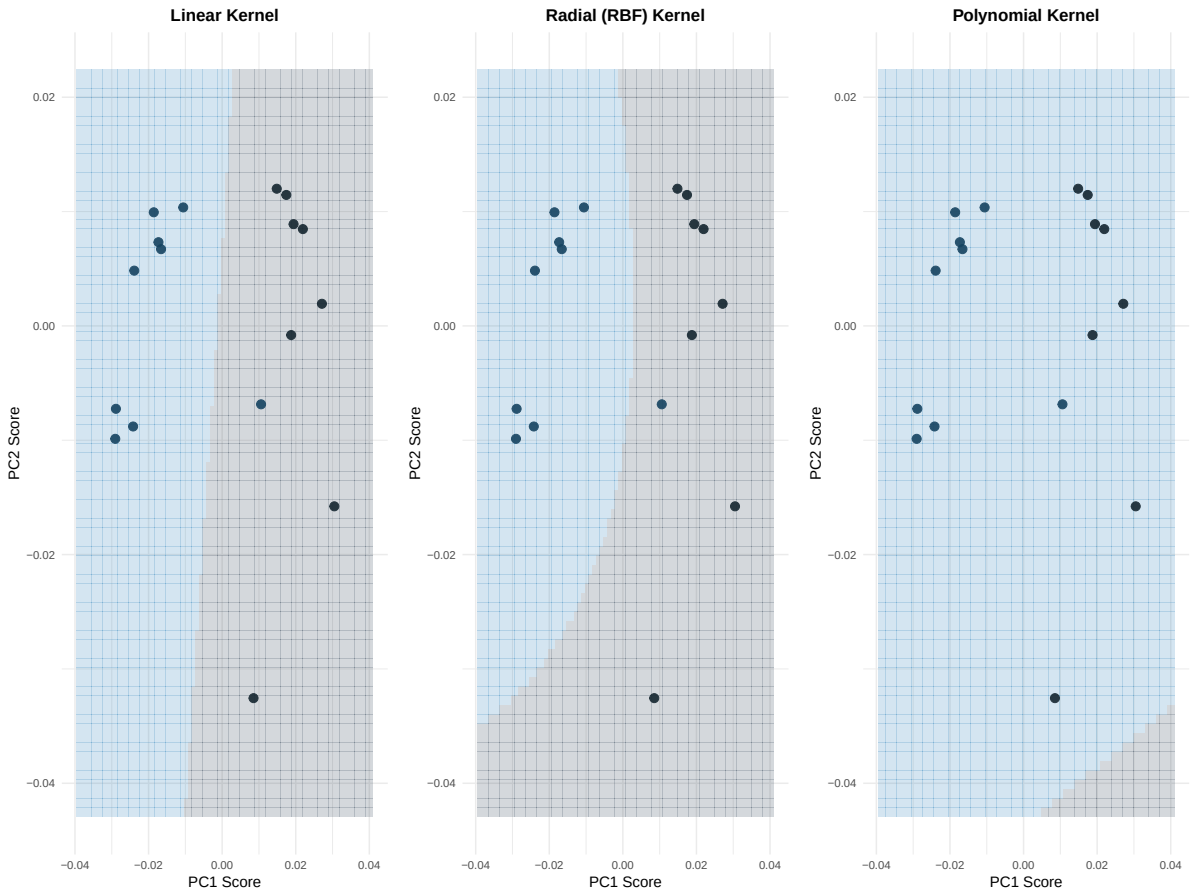
```
# ================================================================
# 2. Supervised Machine Learning (Nested LOOCV for SVM and KNN)
# ================================================================

# ---- Helper: train SVM ----
train_svm <- function(train_x, train_y, test_x, kernel = "radial") {
  train_y_fac <- factor(train_y, levels = c(0, 1))
  maj_class <- as.numeric(names(which.max(table(train_y_fac))))
  maj_prob <- mean(train_y == 1)
  if (length(unique(train_y)) < 2)
    return(list(class = maj_class, prob = maj_prob))

  scaled <- scale_train_test(train_x, test_x)
  tr_x <- scaled$train; te_x <- scaled$test
  tr_x[is.na(tr_x) | is.infinite(tr_x)] <- 0
  te_x[is.na(te_x) | is.infinite(te_x)] <- 0
  gamma_val <- 1 / ncol(tr_x)

  res <- tryCatch({
    model <- svm(tr_x, train_y_fac, kernel = kernel,
                 cost = 1, gamma = gamma_val, scale = FALSE,
                 probability = TRUE)
    pred_obj <- predict(model, te_x, probability = TRUE)
    pred_class <- as.numeric(as.character(pred_obj))
    probs <- attr(pred_obj, "probabilities")
    pred_prob <- extract_prob1(probs)
    list(class = pred_class, prob = pred_prob)
  }, error = function(e) list(class = maj_class, prob = maj_prob))
  return(res)
```

```r
}

# ---- Nested LOOCV for SVM (records kernel used per fold) ----
run_nested_loocv_svm <- function(data, labels, kernels = c("linear", "radial", "polynomial"))
  set.seed(PARAM$seed)
  n <- nrow(data)
  pred_class <- rep(NA, n)
  pred_prob <- rep(NA, n)
  kernel_used <- rep(NA, n)

  for (i in 1:n) {
    train_x <- data[-i, , drop = FALSE]
    train_y <- labels[-i]
    test_x <- data[i, , drop = FALSE]

    # Inner LOOCV to choose best kernel on training set
    inner_acc <- c()
    for (kern in kernels) {
      inner_pred <- rep(NA, n - 1)
      for (j in 1:(n - 1)) {
        inner_train_x <- train_x[-j, , drop = FALSE]
        inner_train_y <- train_y[-j]
        inner_test_x <- train_x[j, , drop = FALSE]
        scaled <- scale_train_test(inner_train_x, inner_test_x)
        tr_x <- scaled$train; te_x <- scaled$test
        tr_x[is.na(tr_x) | is.infinite(tr_x)] <- 0
        te_x[is.na(te_x) | is.infinite(te_x)] <- 0
        gamma_val <- 1 / ncol(tr_x)

        model <- svm(tr_x, as.factor(inner_train_y),
                     kernel = kern, cost = 1, gamma = gamma_val,
                     scale = FALSE, probability = TRUE)
        pred_obj <- predict(model, te_x, probability = TRUE)
        inner_pred[j] <- as.numeric(as.character(pred_obj))
      }
      inner_acc <- c(inner_acc, mean(inner_pred == train_y, na.rm = TRUE))
    }
    best_kernel <- kernels[which.max(inner_acc)]
    kernel_used[i] <- best_kernel
    # Train final model on full training set with best kernel
    scaled_final <- scale_train_test(train_x, test_x)
    final_tr_x <- scaled_final$train
```

```
    final_te_x <- scaled_final$test
    final_tr_x[is.na(final_tr_x) | is.infinite(final_tr_x)] <- 0
    final_te_x[is.na(final_te_x) | is.infinite(final_te_x)] <- 0
    gamma_val_final <- 1 / ncol(final_tr_x)
    final_model <- svm(final_tr_x, as.factor(train_y),
                       kernel = best_kernel, cost = 1, gamma = gamma_val_final,
                       scale = FALSE, probability = TRUE)
    final_pred_obj <- predict(final_model, final_te_x, probability = TRUE)
    pred_class[i] <- as.numeric(as.character(final_pred_obj))
    probs <- attr(final_pred_obj, "probabilities")
    if (!is.null(probs)) {
      pred_prob[i] <- ifelse("1" %in% colnames(probs), probs[1, "1"], probs[1, 1])
    } else {
      pred_prob[i] <- as.numeric(pred_class[i] == 1)
    }
  }
  return(list(pred_class = pred_class, pred_prob = pred_prob, kernel_used = kernel_used))
}

# ---- Helper: train RF ----
train_rf <- function(train_x, train_y, test_x) {
  train_y_fac <- factor(train_y, levels = c(0, 1))
  maj_class <- as.numeric(names(which.max(table(train_y_fac))))
  maj_prob <- mean(train_y == 1)
  if (length(unique(train_y)) < 2)
    return(list(class = maj_class, prob = maj_prob))
  mtry_val <- max(1, floor(ncol(train_x) / 3))
  res <- tryCatch({
    model <- randomForest(train_x, train_y_fac,
                          ntree = PARAM$rf_ntree,
                          mtry = mtry_val)
    pred_class <- as.numeric(as.character(predict(model, test_x)))
    pred_prob <- extract_prob1(predict(model, test_x, type = "prob"))
    list(class = pred_class, prob = pred_prob)
  }, error = function(e) list(class = maj_class, prob = maj_prob))
  return(res)
}

# ---- Simple LOOCV for RF ----
run_loocv_rf <- function(data, labels) {
  n <- nrow(data)
  pred_class <- rep(NA, n)
```

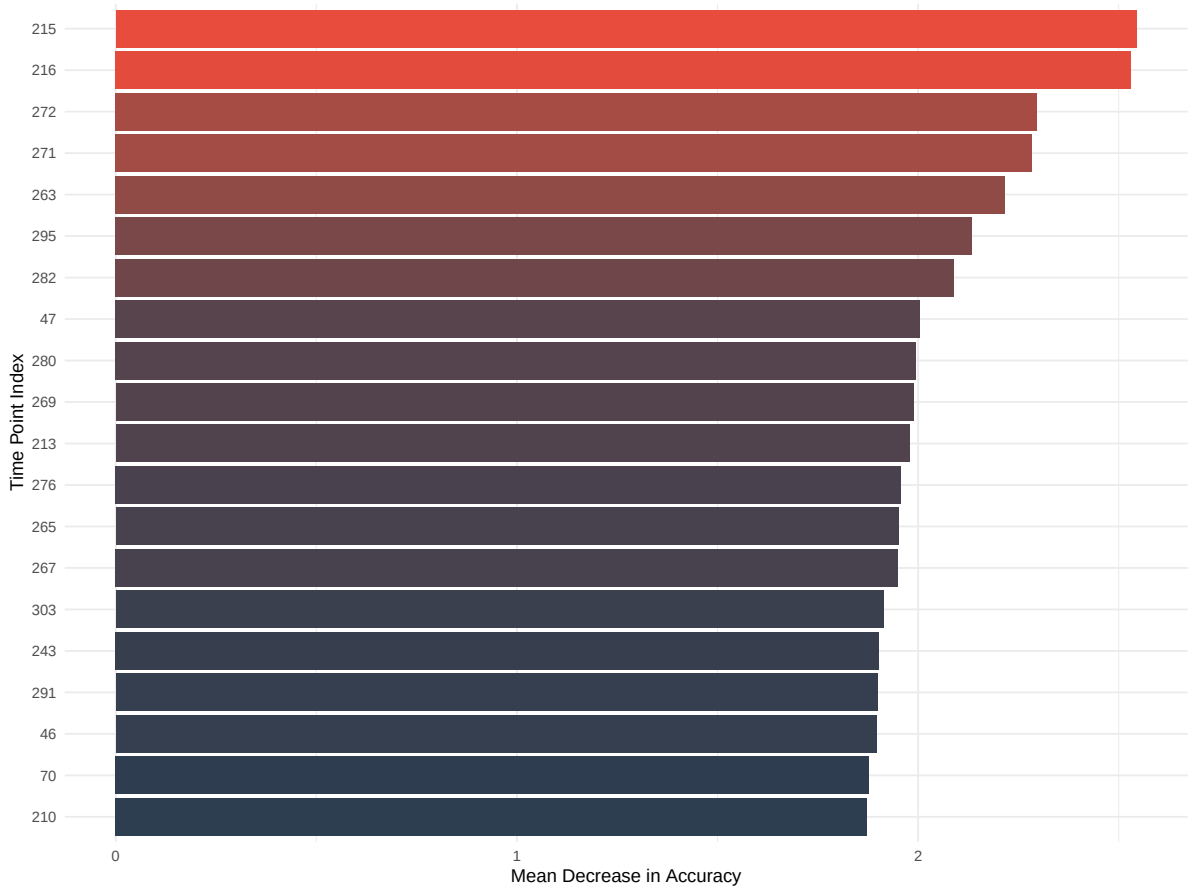
```
  pred_prob <- rep(NA, n)
  for (i in 1:n) {
    train_x <- data[-i, , drop = FALSE]
    train_y <- labels[-i]
    test_x <- data[i, , drop = FALSE]
    result <- train_rf(train_x, train_y, test_x)
    pred_class[i] <- result$class
    pred_prob[i] <- result$prob
  }
  return(list(pred_class = pred_class, pred_prob = pred_prob))
}

# ---- Nested LOOCV for KNN (records K used per fold) ----
run_nested_loocv_knn <- function(data, labels, k_range = 1:15) {
  n <- nrow(data)
  pred_class <- rep(NA, n)
  pred_prob <- rep(NA, n)
  k_used <- rep(NA, n)

  for (i in 1:n) {
    train_x <- data[-i, , drop = FALSE]
    train_y <- labels[-i]
    test_x <- data[i, , drop = FALSE]
    # Inner LOOCV to choose best k on training set
    inner_acc <- c()
    for (k in k_range) {
      inner_pred <- rep(NA, n - 1)
      for (j in 1:(n - 1)) {
        inner_train_x <- train_x[-j, , drop = FALSE]
        inner_train_y <- train_y[-j]
        inner_test_x <- train_x[j, , drop = FALSE]
        scaled <- scale_train_test(inner_train_x, inner_test_x)
        set.seed(PARAM$seed)
        pred <- knn(train = scaled$train, test = scaled$test,
                    cl = factor(inner_train_y), k = k)
        inner_pred[j] <- as.numeric(as.character(pred))
      }
      inner_acc <- c(inner_acc, mean(inner_pred == train_y, na.rm = TRUE))
    }
    best_k <- k_range[which.max(inner_acc)]
    k_used[i] <- best_k
    # Train final model on full training set with best k
```

```
    scaled_final <- scale_train_test(train_x, test_x)
    set.seed(PARAM$seed)
    final_pred <- knn(train = scaled_final$train, test = scaled_final$test,
                      cl = factor(train_y), k = best_k, prob = TRUE)
    pred_class[i] <- as.numeric(as.character(final_pred))
    prob_val <- attr(final_pred, "prob")
    pred_prob[i] <- if (pred_class[i] == 1) prob_val else 1 - prob_val
  }
  return(list(pred_class = pred_class, pred_prob = pred_prob, k_used = k_used))
}

# ================================================================
# 2a. Diagnostic Visualizations
# ================================================================

# ---- SVM Decision Boundary Visualization (using PC1/PC2) ----
if (ncol(pca_scores) >= 2) {
  p_svm_boundaries <- plot_svm_boundaries(pca_scores[, 1:2, drop = FALSE], y_true)
  print(p_svm_boundaries)
} else {
  cat("Warning: Only 1 PC retained, cannot produce 2D SVM boundary plot.\n")
}
```




```r
# ---- RF Variable Importance (Trained on Full Data for Interpretation) ----
set.seed(PARAM$seed)
rf_imp_model <- randomForest(
  x = DataBoth,
  y = as.factor(y_true),
  ntree = PARAM$rf_ntree,
  importance = TRUE,
  mtry = max(1, floor(ncol(DataBoth) / 3))
)

imp_matrix <- importance(rf_imp_model)
if ("MeanDecreaseAccuracy" %in% colnames(imp_matrix)) {
  imp_col <- "MeanDecreaseAccuracy"
} else if ("%IncMSE" %in% colnames(imp_matrix)) {
  imp_col <- "%IncMSE"
} else {
  imp_col <- colnames(imp_matrix)[1]
```

```r
}

var_imp <- data.frame(
  TimePoint = 1:ncol(DataBoth),
  Importance = imp_matrix[, imp_col]
)

top20 <- var_imp |>
  arrange(desc(Importance)) |>
  slice(1:20) |>
  mutate(TimePoint = as.numeric(TimePoint))

p_rf_imp <- ggplot(top20, aes(x = reorder(TimePoint, Importance),
                              y = Importance, fill = Importance)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  scale_fill_gradient(low = "#2C3E50", high = "#E74C3C") +
  labs(title = " ", #Top 20 Important Time Points (Random Forest)
       x = "Time Point Index",
       y = "Mean Decrease in Accuracy") +
  theme_minimal(base_size = 10) +
  theme(legend.position = "none")
print(p_rf_imp)
```




```
# ================================================================
# 2b. Run all classifiers (all use LOOCV evaluation)
# ================================================================

# SVM: Nested LOOCV (kernel selected inside each fold)
svm_raw <- run_nested_loocv_svm(DataBoth, y_true)
svm_pca <- run_nested_loocv_svm(pca_scores, y_true)

# RF: Simple LOOCV (fixed hyperparameters)
rf_raw <- run_loocv_rf(DataBoth, y_true)
rf_pca <- run_loocv_rf(pca_scores, y_true)

# KNN: Nested LOOCV (k selected inside each fold)
knn_raw <- run_nested_loocv_knn(DataBoth, y_true, PARAM$knn_k_range)
knn_pca <- run_nested_loocv_knn(pca_scores, y_true, PARAM$knn_k_range)

# Extract hyperparameters selected per fold
```

```r
kernel_used_raw <- svm_raw$kernel_used
kernel_used_pca <- svm_pca$kernel_used
k_used_raw <- knn_raw$k_used
k_used_pca <- knn_pca$k_used

# ---- Compute accuracy ----
svm_raw_acc <- round(calc_metrics(svm_raw$pred_class, y_true)["Accuracy"], 3)
svm_pca_acc <- round(calc_metrics(svm_pca$pred_class, y_true)["Accuracy"], 3)
rf_raw_acc   <- round(calc_metrics(rf_raw$pred_class, y_true)["Accuracy"], 3)
rf_pca_acc   <- round(calc_metrics(rf_pca$pred_class, y_true)["Accuracy"], 3)
knn_raw_acc  <- round(calc_metrics(knn_raw$pred_class, y_true)["Accuracy"], 3)
knn_pca_acc  <- round(calc_metrics(knn_pca$pred_class, y_true)["Accuracy"], 3)

cat("\n--- LOOCV Accuracy Results ---\n")
```

```
--- LOOCV Accuracy Results ---
```

```r
cat(sprintf("SVM Raw: %.3f, SVM FPCA: %.3f\n", svm_raw_acc, svm_pca_acc))
```

```
SVM Raw: 0.882, SVM FPCA: 0.765
```

```r
cat(sprintf("RF  Raw: %.3f, RF  FPCA: %.3f\n", rf_raw_acc, rf_pca_acc))
```

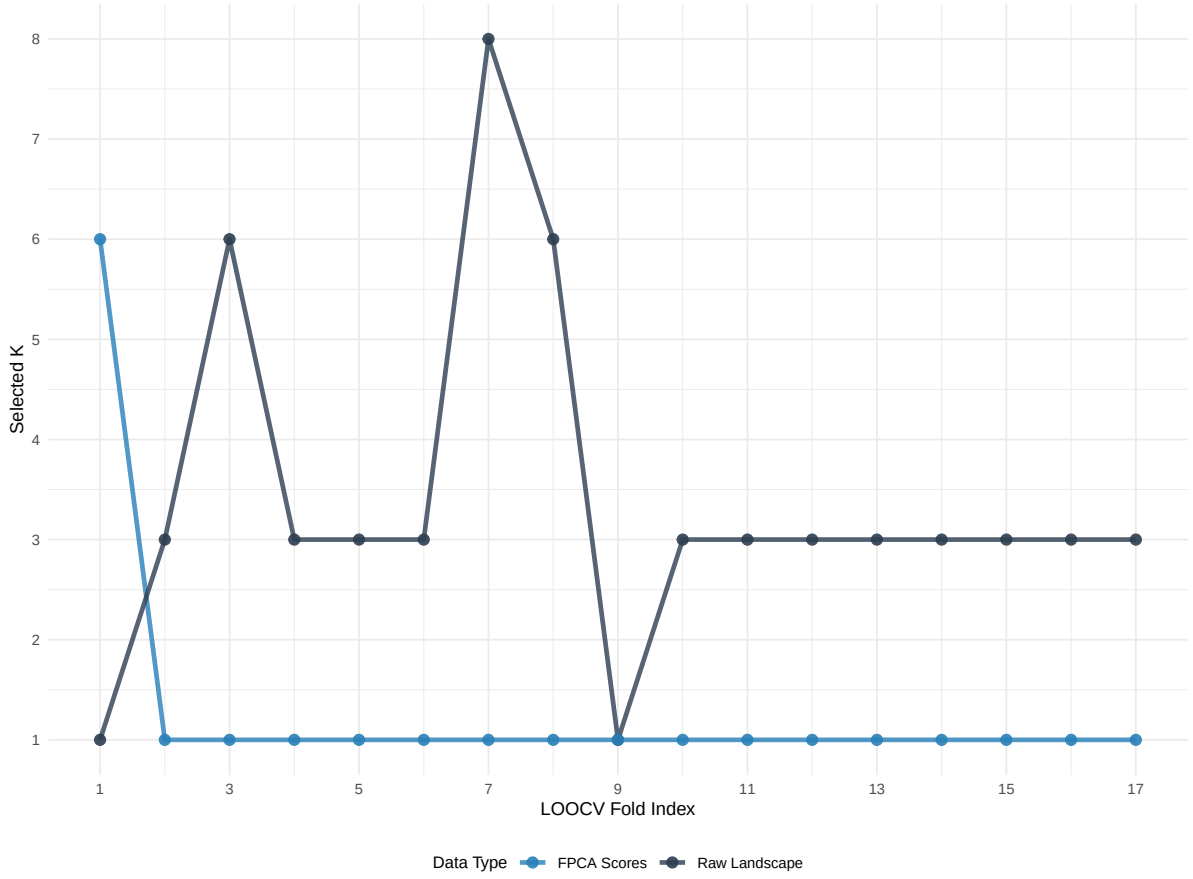
```
RF  Raw: 0.882, RF  FPCA: 0.882
```

```r
cat(sprintf("KNN Raw: %.3f, KNN FPCA: %.3f\n", knn_raw_acc, knn_pca_acc))
```

```
KNN Raw: 0.882, KNN FPCA: 0.882
```

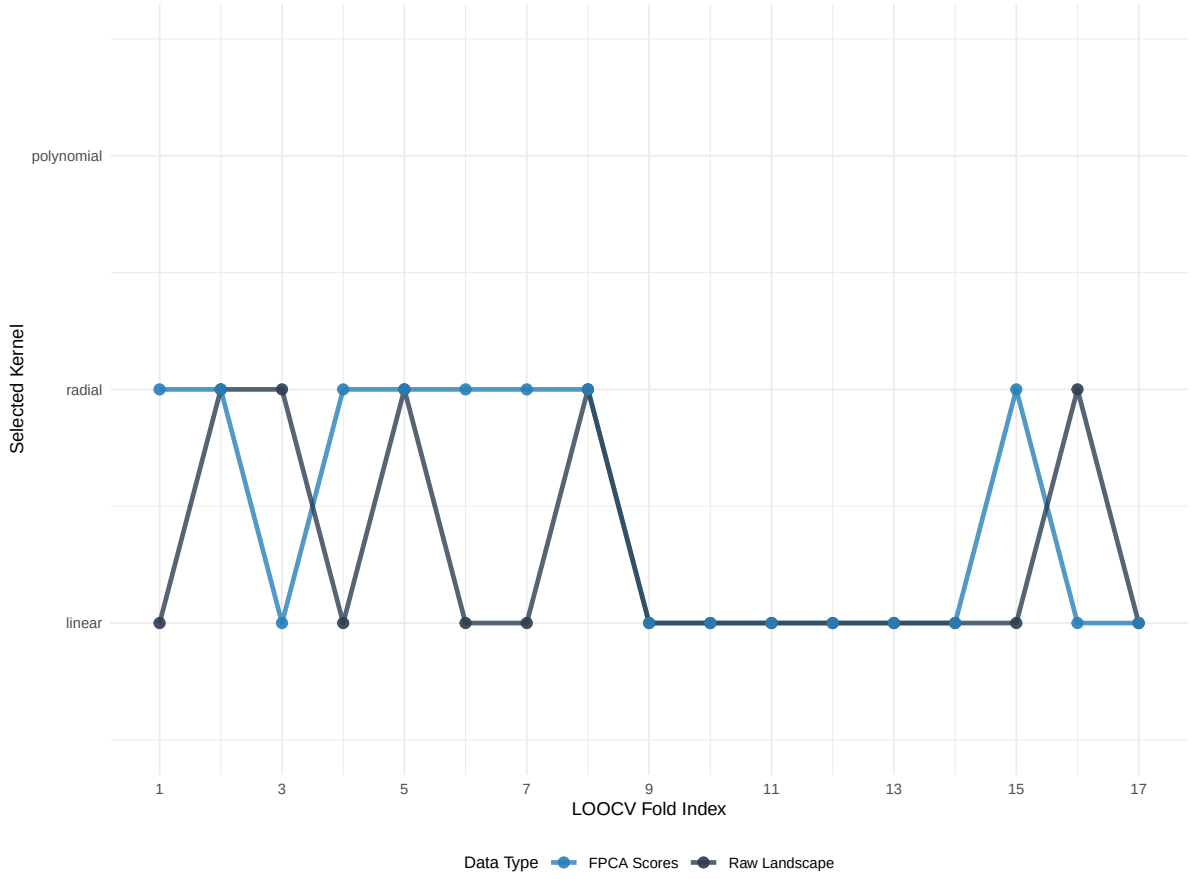
```r
# ================================================================
# 2c. Hyperparameter Selection Across Folds
# ================================================================

# ---- KNN K-value Across Folds ----
k_trend_df <- data.frame(
  Fold = rep(1:length(k_used_raw), 2), K = c(k_used_raw, k_used_pca),
  Data = rep(c("Raw Landscape", "FPCA Scores"),
```

```
              each = length(k_used_raw))
)

p_knn_trend <- ggplot(k_trend_df, aes(x = Fold, y = K, color = Data, group = Data)) +
  geom_line(linewidth = 1.2, alpha = 0.8) +
  geom_point(size = 2.5, alpha = 0.9) +
  scale_color_manual(values = c("Raw Landscape" = "#2C3E50",
                                "FPCA Scores" = "#2980B9")) +
  scale_x_continuous(breaks = seq(1, length(k_used_raw), by = 2)) +
  scale_y_continuous(breaks = 1:max(PARAM$knn_k_range)) +
  labs(
    title = " ", # KNN K-Value Selected per LOOCV Fold
    x = "LOOCV Fold Index", y = "Selected K",
    color = "Data Type"
  ) +
  theme_minimal(base_size = 10) +
  theme(
    plot.title = element_text(face = "bold", size = 11),
    legend.position = "bottom",
    legend.title = element_text(size = 9)
  )

print(p_knn_trend)
```

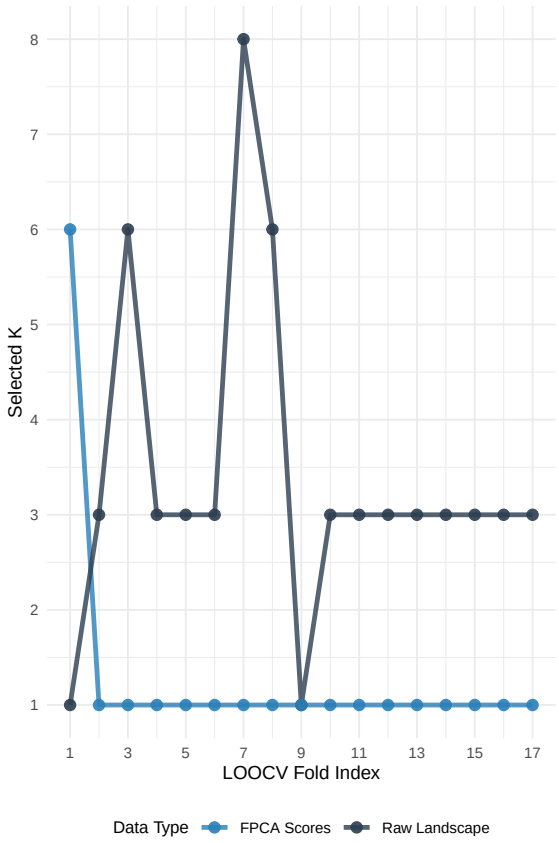


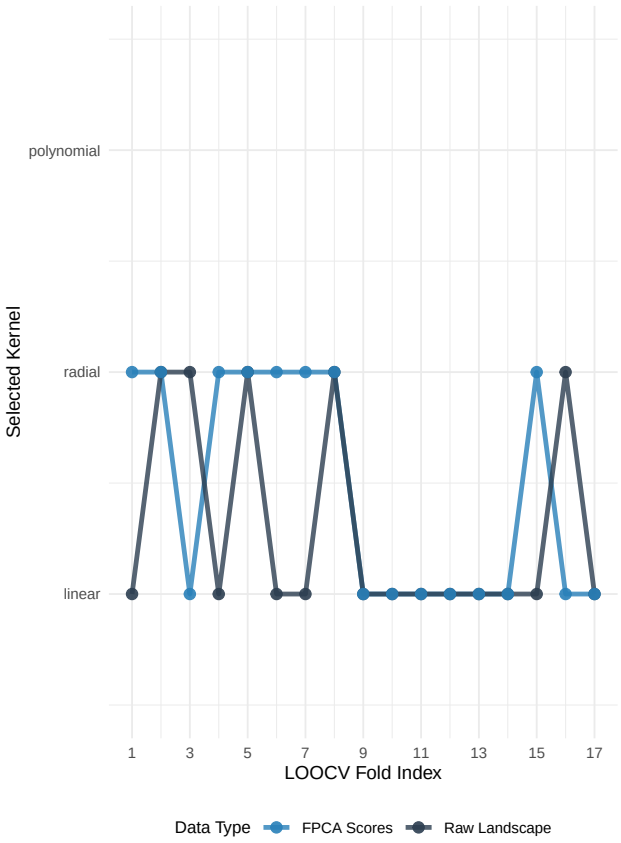

```
# ---- SVM Kernel Across Folds ----
kernel_numeric <- c("linear" = 1, "radial" = 2, "polynomial" = 3)

kernel_trend_df <- data.frame(
  Fold = rep(1:length(kernel_used_raw), 2),
  Kernel = c(kernel_used_raw, kernel_used_pca),
  KernelNumeric = c(as.numeric(factor(kernel_used_raw,
                                      levels = c("linear", "radial", "polynomial"))),
                    as.numeric(factor(kernel_used_pca,
                                      levels = c("linear", "radial","polynomial")))),
  Data = rep(c("Raw Landscape", "FPCA Scores"),
             each = length(kernel_used_raw))
)

p_svm_trend <- ggplot(kernel_trend_df, aes(x = Fold, y = KernelNumeric,
                                           color = Data, group = Data)) +
  geom_line(linewidth = 1.2, alpha = 0.8) +
```

```
  geom_point(size = 2.5, alpha = 0.9) +
  scale_color_manual(values = c("Raw Landscape" = "#2C3E50",
                                "FPCA Scores" = "#2980B9")) +
  scale_x_continuous(breaks = seq(1, length(kernel_used_raw), by = 2)) +
  scale_y_continuous(
    breaks = c(1, 2, 3),
    labels = c("linear", "radial", "polynomial"),
    limits = c(0.5, 3.5)
  ) +
  labs(
    title = " ", # SVM Kernel Selected per LOOCV Fold
    x = "LOOCV Fold Index", y = "Selected Kernel",
    color = "Data Type"
  ) +
  theme_minimal(base_size = 10) +
  theme(
    plot.title = element_text(face = "bold", size = 11),
    legend.position = "bottom",
    legend.title = element_text(size = 9)
  )

print(p_svm_trend)
```



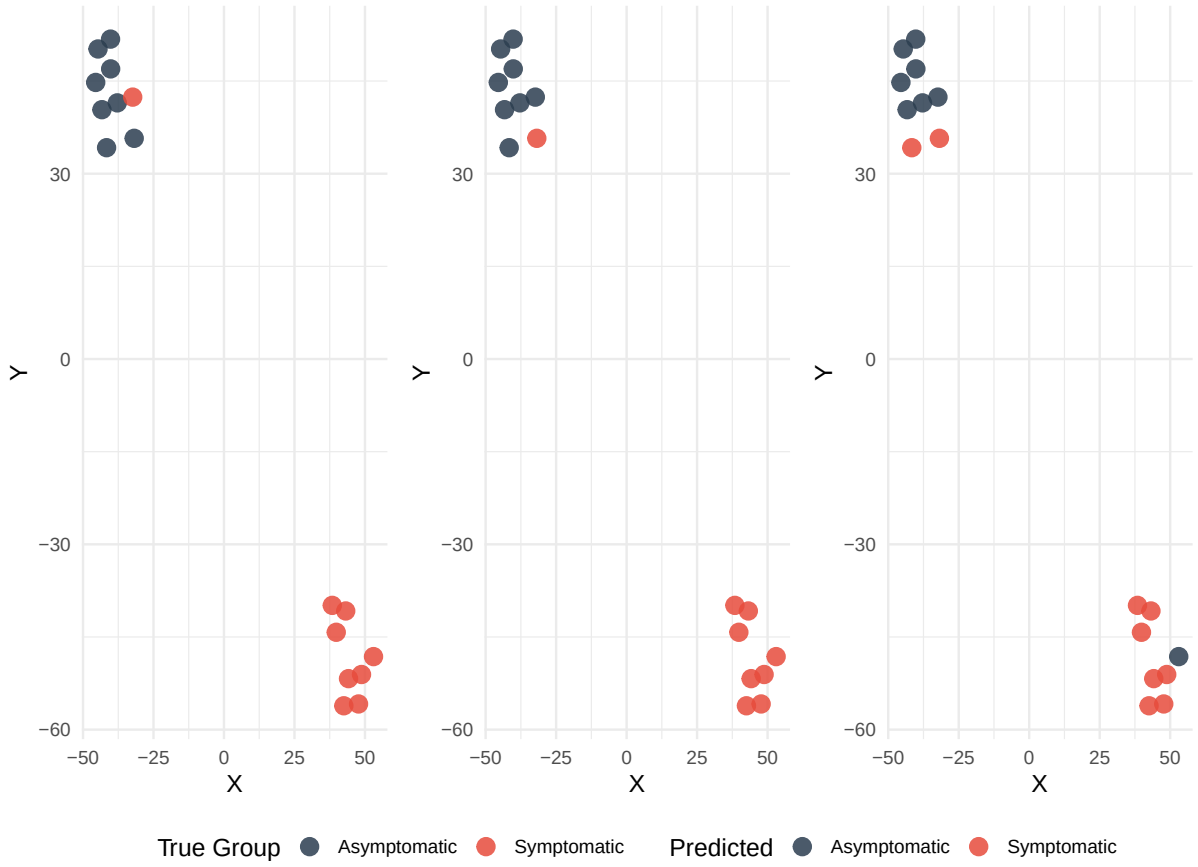

```
# ---- Combined Grid ----
hyperparam_grid <- wrap_plots(p_knn_trend, p_svm_trend, ncol = 2) +
  plot_annotation(
    title = " ", # Nested LOOCV Hyperparameter Selection Across Folds
#    subtitle = "Each fold independently selects optimal hyperparameters via inner cross-vali
    theme = theme(plot.title = element_text(size = 12, face = "bold"),
                  plot.subtitle = element_text(size = 9, color = "grey40"))
  )

print(hyperparam_grid)
```



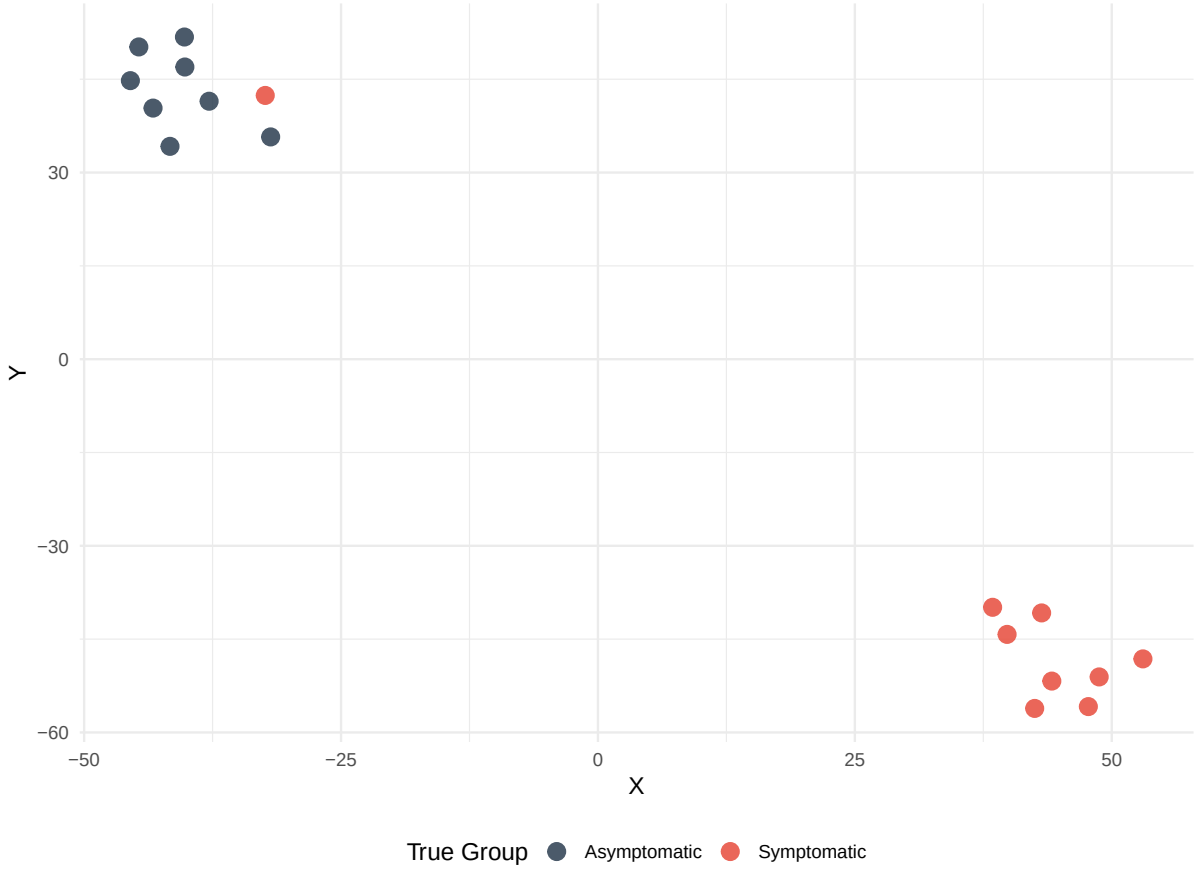



```
# ================================================================
# 2d. Prediction Visualizations (SVM, RF, KNN)
# ================================================================

p_gt_class <- ggplot(data.frame(X = proj_coords$X, Y = proj_coords$Y,
                                Group = factor(ifelse(y_true == 1, "Symptomatic", "Asymptomatic")))
               aes(x = X, y = Y, color = Group)) +
  geom_point(size = 4, alpha = 0.85) +
  scale_color_manual(values = c("Asymptomatic" = "#2C3E50", "Symptomatic" = "#E74C3C")) +
  labs(title = " ", #Ground Truth
 #      subtitle = proj_label,
       color = "True Group") +
  theme_minimal(base_size = 13) + theme(legend.position = "bottom")

# ---- SVM Predictions ----
p_svm_raw <- ggplot(data.frame(X = proj_coords$X, Y = proj_coords$Y,
                               Pred = factor(ifelse(svm_raw$pred_class == 1, "Symptomatic",
```

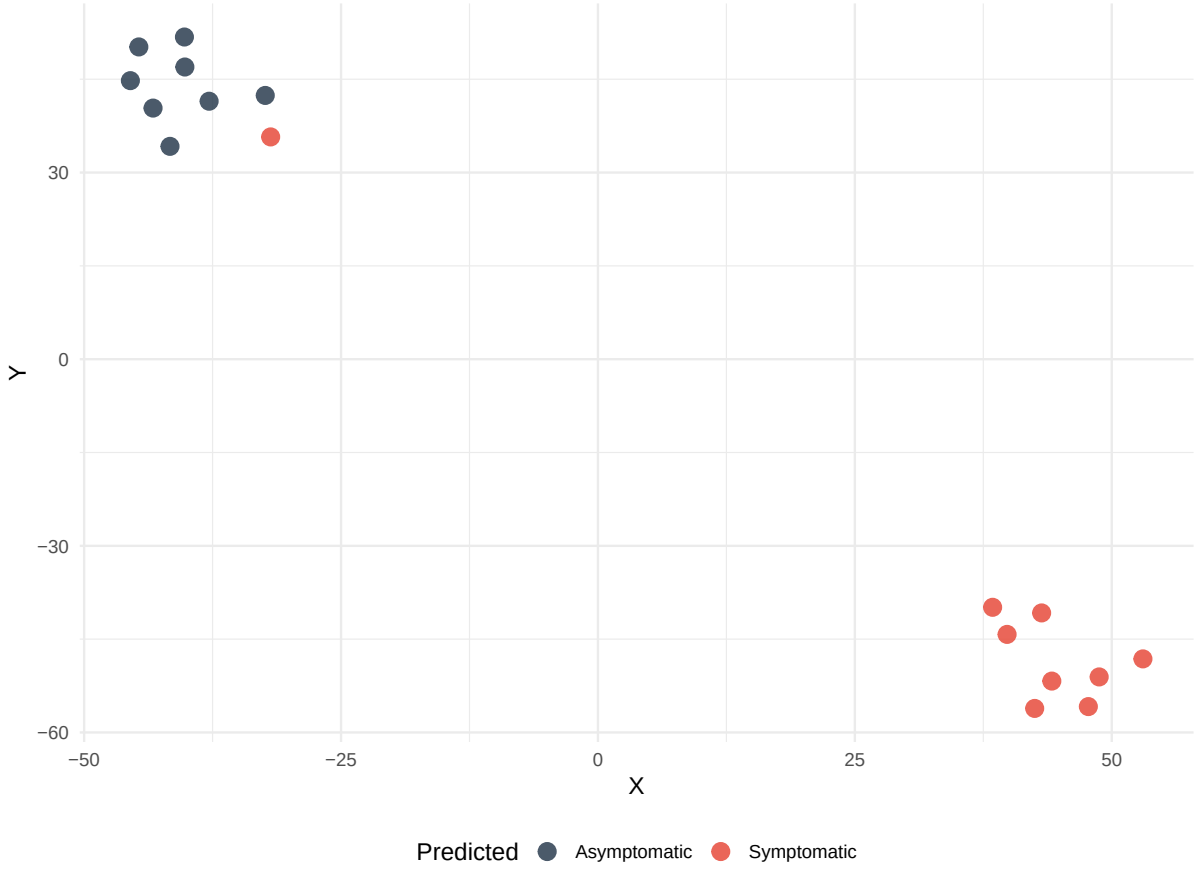
```
                                                levels = c("Asymptomatic", "Symptomatic"))),
                    aes(x = X, y = Y, color = Pred)) +
  geom_point(size = 4, alpha = 0.85) +
  scale_color_manual(values = c("Asymptomatic" = "#2C3E50", "Symptomatic" = "#E74C3C")) +
  labs(title = " ", #SVM Predictions (Raw Data)
       # subtitle = paste("LOOCV Accuracy =", svm_raw_acc, " (", proj_label, ")"),
       color = "Predicted") +
  theme_minimal(base_size = 13) + theme(legend.position = "bottom")

p_svm_pca <- ggplot(data.frame(X = proj_coords$X, Y = proj_coords$Y,
                               Pred = factor(ifelse(svm_pca$pred_class == 1, "Symptomatic",
                                                levels = c("Asymptomatic", "Symptomatic"))),
                    aes(x = X, y = Y, color = Pred)) +
  geom_point(size = 4, alpha = 0.85) +
  scale_color_manual(values = c("Asymptomatic" = "#2C3E50", "Symptomatic" = "#E74C3C")) +
  labs(title = " ", #SVM Predictions (FPCA Scores)
      # subtitle = paste("LOOCV Accuracy =", svm_pca_acc, " (", proj_label, ")"),
       color = "Predicted") +
  theme_minimal(base_size = 13) + theme(legend.position = "bottom")

svm_grid <- (wrap_plots(p_gt_class, p_svm_raw, p_svm_pca, ncol = 3, guides = "collect") &
               theme(legend.position = "bottom")
             #&
              # plot_annotation(title = paste0("SVM LOOCV Predictions (", proj_label, #")"))
               #theme(plot.title = element_text(size = 10, face = "bold"))
               )
print(svm_grid)
```



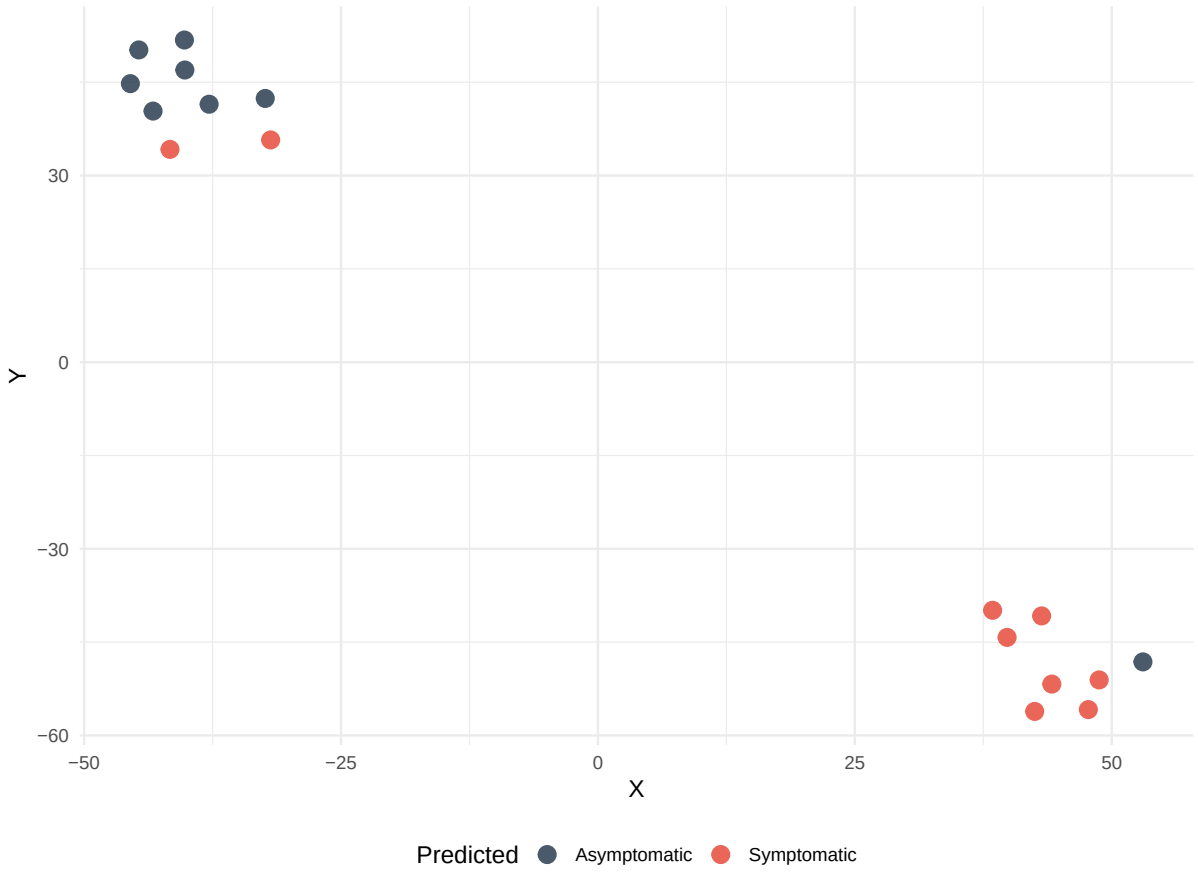

```
print(p_gt_class)
```




```
print(p_svm_raw)
```

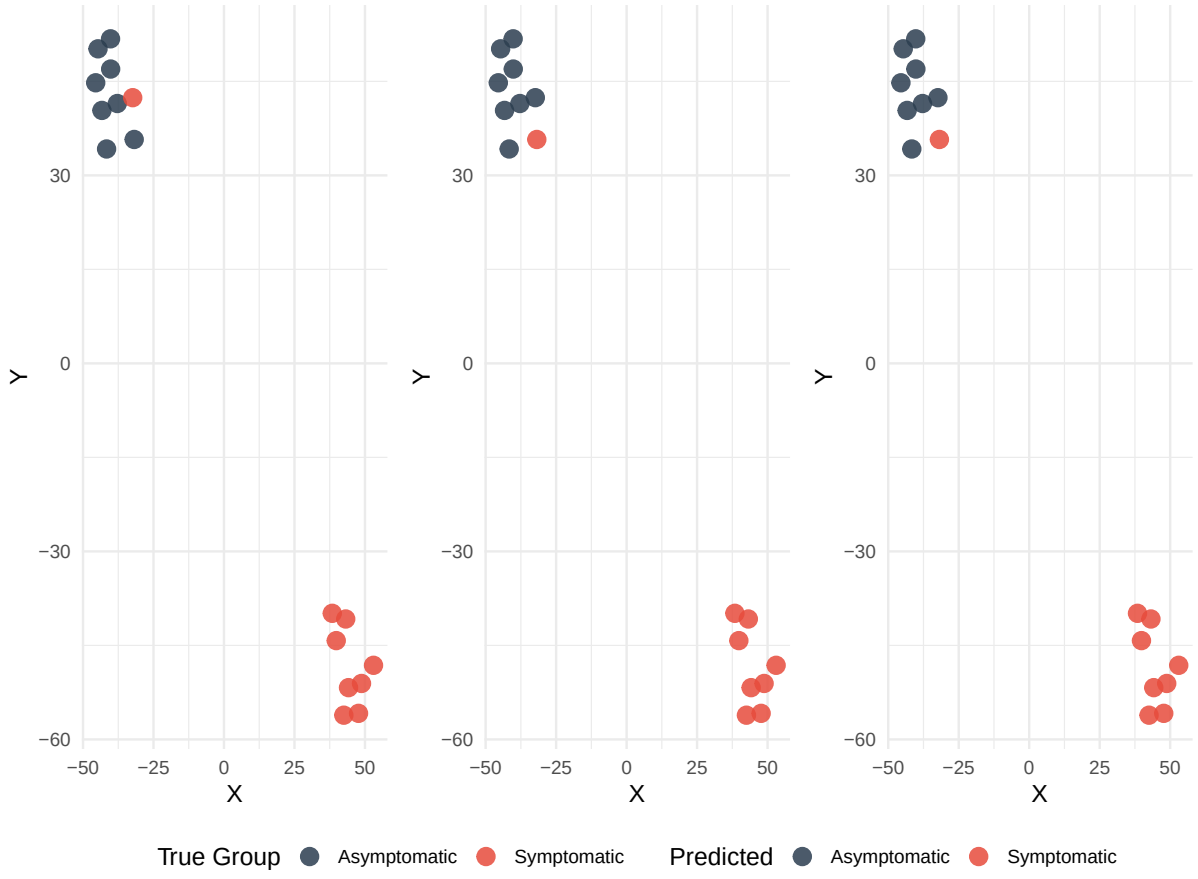



```
print(p_svm_pca)
```

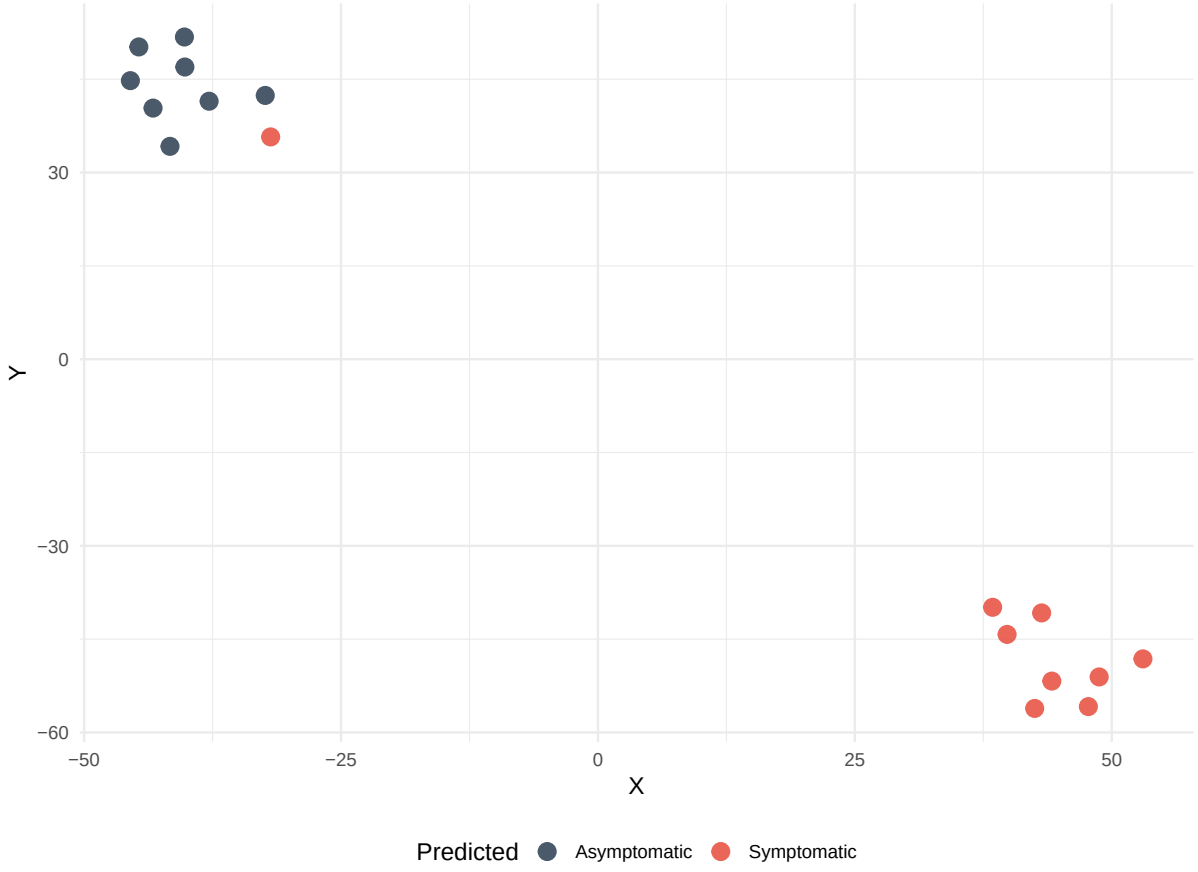



```
p_svm_raw <- p_svm_raw + theme(aspect.ratio = 1.8)
p_svm_pca <- p_svm_pca + theme(aspect.ratio = 1.8)

# ---- RF Predictions ----
p_rf_raw <- ggplot(data.frame(X = proj_coords$X, Y = proj_coords$Y,
                              Pred = factor(ifelse(rf_raw$pred_class == 1,
                                                   "Symptomatic", "Asymptomatic"),
                                            levels = c("Asymptomatic",
                                                       "Symptomatic"))),
                   aes(x = X, y = Y, color = Pred)) +
  geom_point(size = 4, alpha = 0.85) +
  scale_color_manual(values = c("Asymptomatic" = "#2C3E50",
                                "Symptomatic" = "#E74C3C")) +
  labs(title = " ", #RF Predictions (Raw Data)
  #     subtitle = paste("LOOCV Accuracy =", rf_raw_acc, " (", proj_label, ")"),
       color = "Predicted") +
  theme_minimal(base_size = 13) + theme(legend.position = "bottom")
```

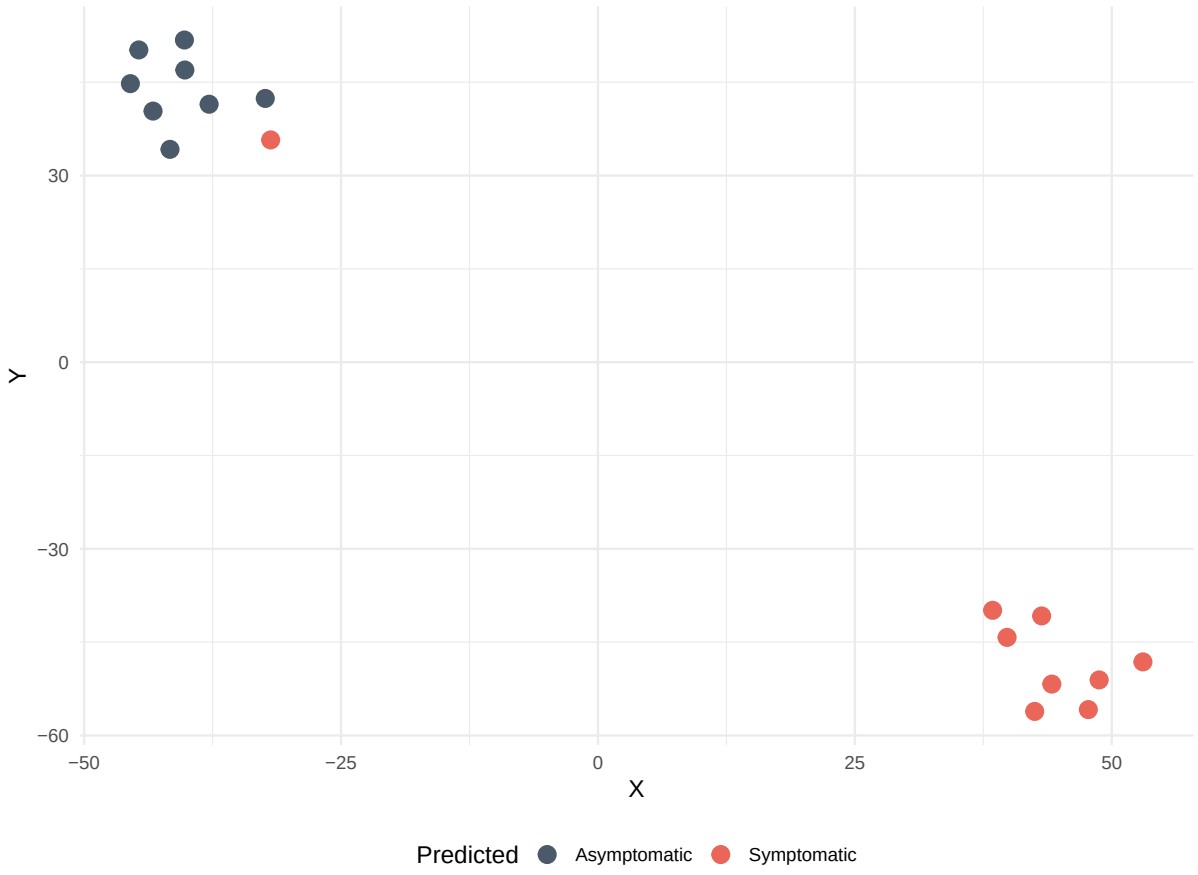
```
p_rf_pca <- ggplot(data.frame(X = proj_coords$X, Y = proj_coords$Y,
                              Pred = factor(ifelse(rf_pca$pred_class == 1,
                                                   "Symptomatic", "Asymptomatic"),
                                            levels = c("Asymptomatic","Symptomatic"))),
                   aes(x = X, y = Y, color = Pred)) +
  geom_point(size = 4, alpha = 0.85) +
  scale_color_manual(values = c("Asymptomatic" = "#2C3E50", "Symptomatic" = "#E74C3C")) +
  labs(title = " ", #RF Predictions (FPCA Scores)
 #      subtitle = paste("LOOCV Accuracy =", rf_pca_acc,
                       # " (", proj_label, ")"),
 color = "Predicted"
 ) +
  theme_minimal(base_size = 13) + theme(legend.position = "bottom")

rf_grid <- (wrap_plots(p_gt_class, p_rf_raw, p_rf_pca, ncol = 3,
                       guides = "collect") &
              theme(legend.position = "bottom") &
              #plot_annotation(title = paste0("Random Forest LOOCV Predictions (",
               #                               proj_label, ")")) &
              theme(plot.title = element_text(size = 10, face = "bold")))
print(rf_grid)
```




```
print(p_rf_raw)
```

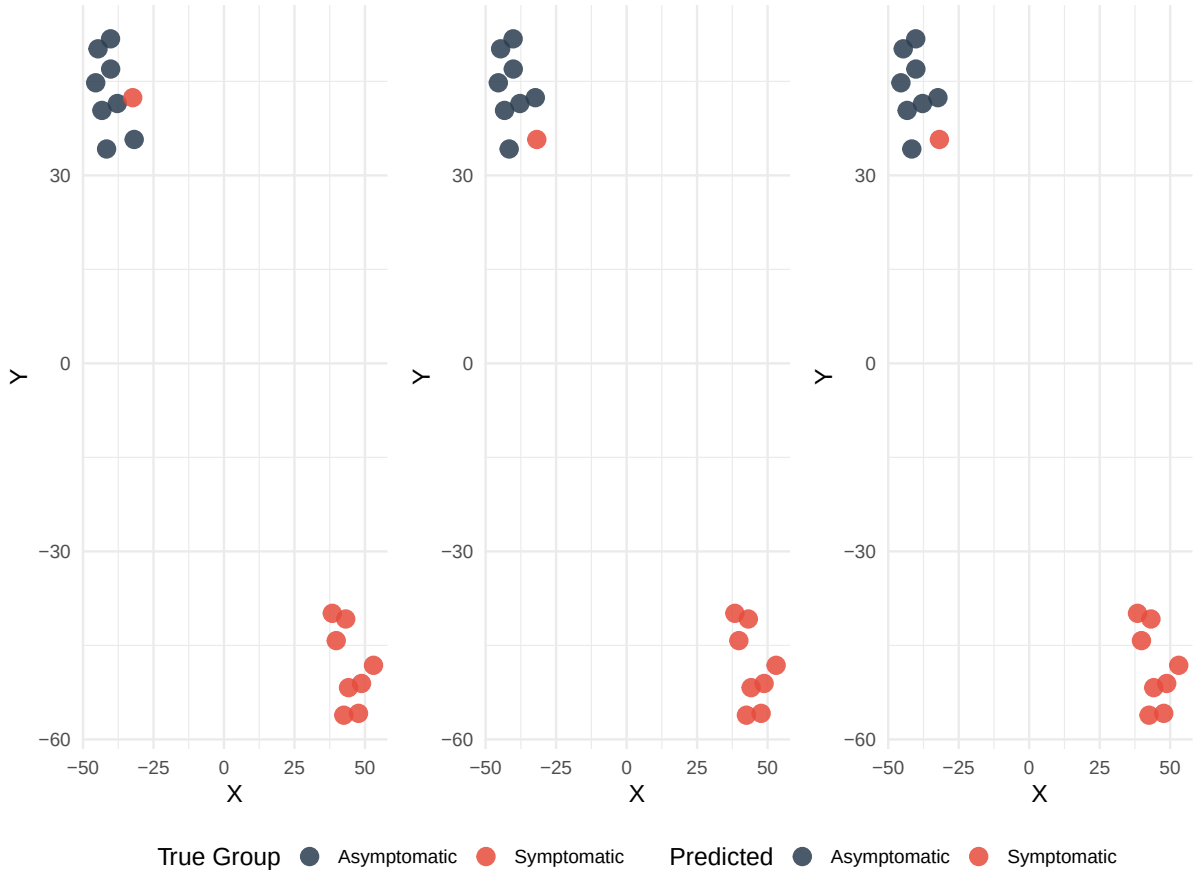



```
print(p_rf_pca)
```

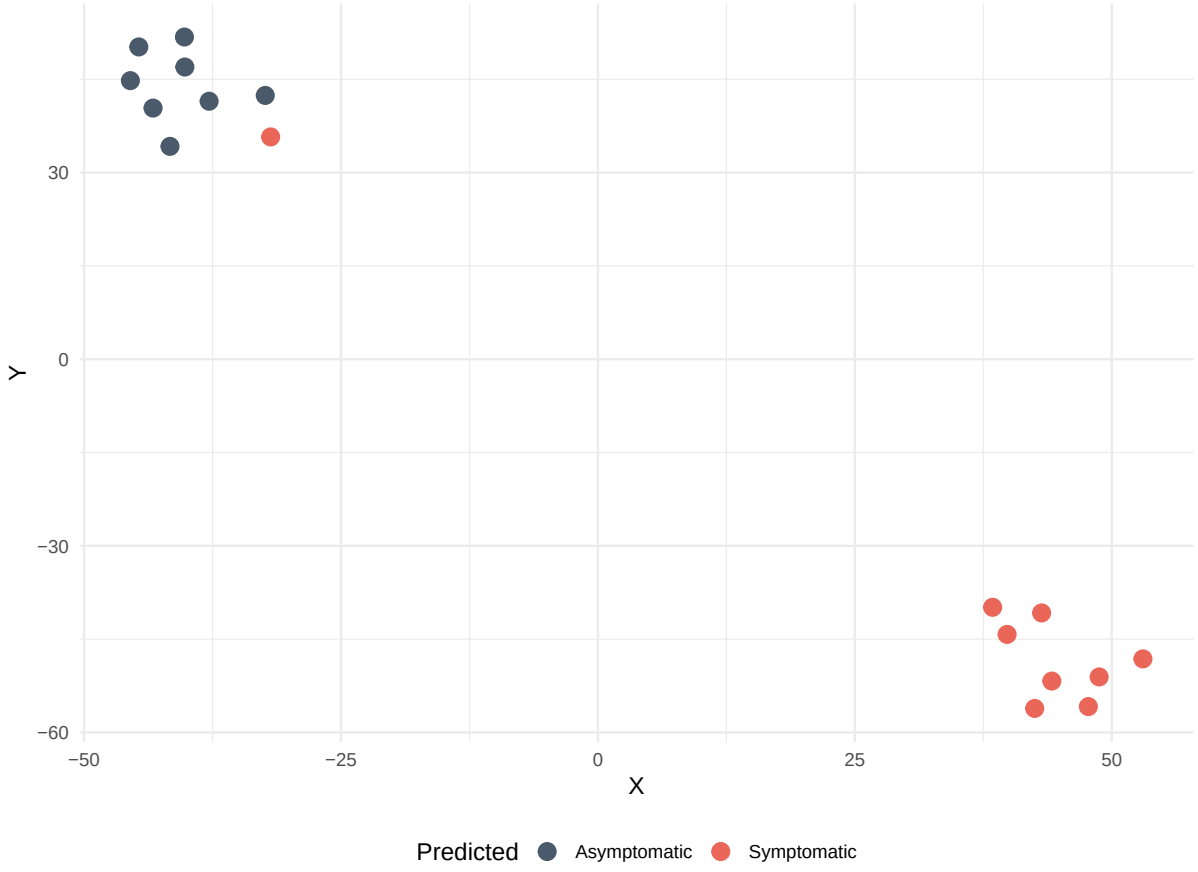



```
p_rf_raw <- p_rf_raw + theme(aspect.ratio = 1.8)
p_rf_pca <- p_rf_pca + theme(aspect.ratio = 1.8)

# ---- KNN Predictions ----
p_knn_raw <- ggplot(data.frame(X = proj_coords$X,
                               Y = proj_coords$Y,
                               Pred = factor(ifelse(knn_raw$pred_class == 1,
                                                    "Symptomatic", "Asymptomatic"),
                               levels = c("Asymptomatic", "Symptomatic"))),
                    aes(x = X, y = Y, color = Pred)) +
  geom_point(size = 4, alpha = 0.85) +
  scale_color_manual(values = c("Asymptomatic" = "#2C3E50", "Symptomatic" = "#E74C3C")) +
  labs(title = " ", #KNN Predictions (Raw Data)
       #subtitle = paste("LOOCV Accuracy =", knn_raw_acc, " (", proj_label, ")"),
       color = "Predicted") +
  theme_minimal(base_size = 13) + theme(legend.position = "bottom")
```

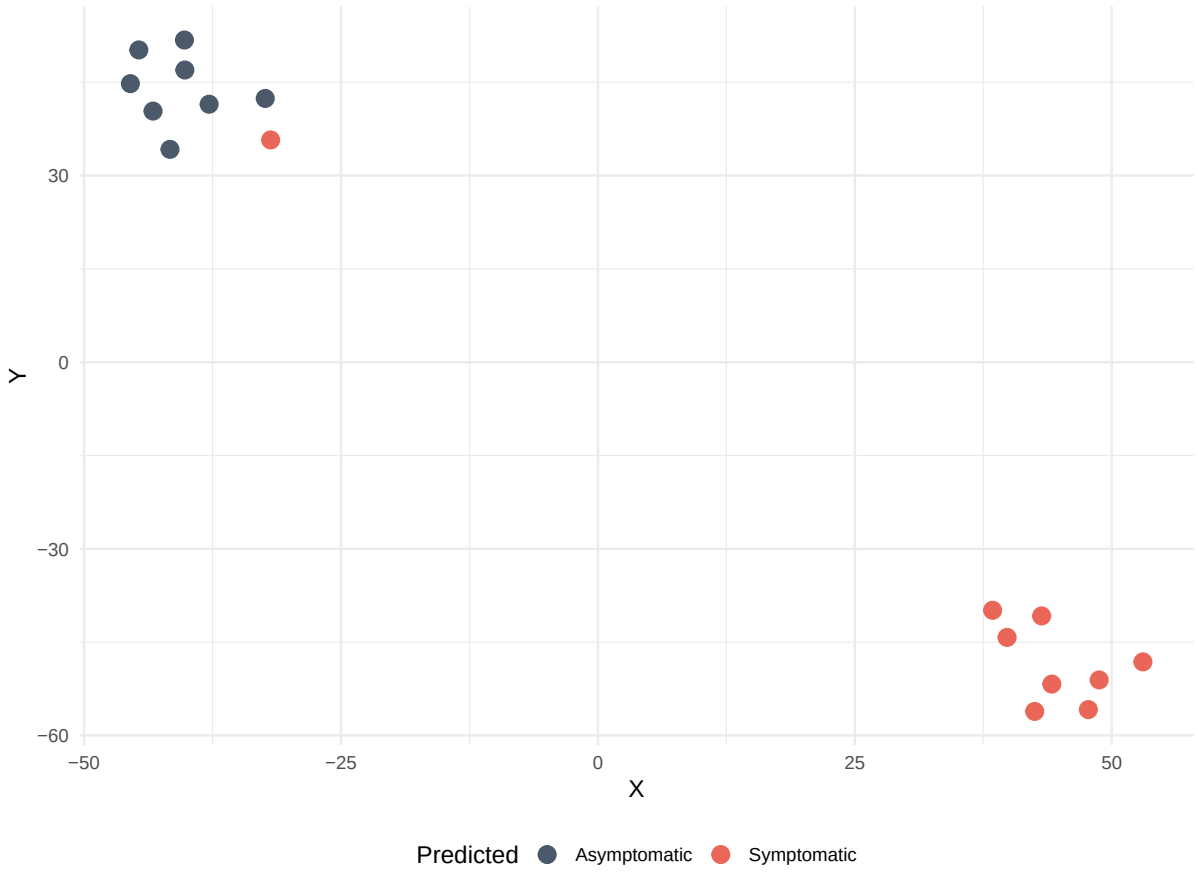
```r
p_knn_pca <- ggplot(data.frame(X = proj_coords$X, Y = proj_coords$Y,
                               Pred = factor(ifelse(knn_pca$pred_class == 1,
                                                    "Symptomatic", "Asymptomatic"),
                                             levels = c("Asymptomatic", "Symptomatic"))),
                    aes(x = X, y = Y, color = Pred)) +
  geom_point(size = 4, alpha = 0.85) +
  scale_color_manual(values = c("Asymptomatic" = "#2C3E50",
                                "Symptomatic" = "#E74C3C")) +
  labs(title = " ", #KNN Predictions (FPCA Scores)
      # subtitle = paste("LOOCV Accuracy =", knn_pca_acc,
                         # " (", proj_label, ")"),
       color = "Predicted") +
  theme_minimal(base_size = 13) + theme(legend.position = "bottom")

knn_grid <- (wrap_plots(p_gt_class, p_knn_raw, p_knn_pca, ncol = 3,
                        guides = "collect") &
               theme(legend.position = "bottom") &
               #plot_annotation(title = paste0("KNN LOOCV Predictions (",
                                              # proj_label, ")")) &
               theme(plot.title = element_text(size = 10, face = "bold")))
print(knn_grid)
```




```
print(p_knn_raw)
```




```
print(p_knn_pca)
```




```
p_gt_class <- p_gt_class + theme(aspect.ratio = 1.8)
p_knn_raw <- p_knn_raw + theme(aspect.ratio = 1.8)
p_knn_pca <- p_knn_pca + theme(aspect.ratio = 1.8)

# ================================================================
# 2e. Performance Metrics and ROC Curves
# ================================================================

format_results <- function(pred_class, pred_prob, true) {
  m_res <- calc_metrics(pred_class, true)
  valid_idx <- !is.na(true) & !is.na(pred_prob)
  true_clean <- true[valid_idx]
  prob_clean <- pred_prob[valid_idx]
  has_both <- length(unique(true_clean)) == 2 && all(c(0, 1) %in% unique(true_clean))
  roc_obj <- NULL; auc_val <- NA
  if (has_both) {
    roc_obj <- tryCatch(roc(true_clean, prob_clean, levels = c(0, 1), quiet = TRUE),
```

```
                          error = function(e) NULL)
    if (!is.null(roc_obj)) auc_val <- as.numeric(auc(roc_obj))
  }
  if (is.null(roc_obj)) roc_obj <- list(sensitivities = c(0, 1), specificities = c(1, 0))
  metrics_df <- data.frame(
    Accuracy = round(m_res["Accuracy"], 3),
    Balanced_Accuracy = round(m_res["Balanced_Accuracy"], 3),
    Sensitivity = round(m_res["Sensitivity"], 3),
    Specificity = round(m_res["Specificity"], 3),
    Precision = round(m_res["Precision"], 3),
    F1 = round(m_res["F1"], 3),
    MCC = round(m_res["MCC"], 3),
    AUC = ifelse(is.na(auc_val), NA, round(auc_val, 3))
  )
  return(list(metrics = metrics_df, roc_obj = roc_obj))
}

results <- list(
  SVM = list(
    Raw = c(list(pred = svm_raw$pred_class, prob = svm_raw$pred_prob),
            format_results(svm_raw$pred_class, svm_raw$pred_prob, y_true)),
    FPCA = c(list(pred = svm_pca$pred_class, prob = svm_pca$pred_prob),
             format_results(svm_pca$pred_class, svm_pca$pred_prob, y_true))
  ),
  RF = list(
    Raw = c(list(pred = rf_raw$pred_class, prob = rf_raw$pred_prob),
            format_results(rf_raw$pred_class, rf_raw$pred_prob, y_true)),
    FPCA = c(list(pred = rf_pca$pred_class, prob = rf_pca$pred_prob),
             format_results(rf_pca$pred_class, rf_pca$pred_prob, y_true))
  ),
  KNN = list(
    Raw = c(list(pred = knn_raw$pred_class, prob = knn_raw$pred_prob),
            format_results(knn_raw$pred_class, knn_raw$pred_prob, y_true)),
    FPCA = c(list(pred = knn_pca$pred_class, prob = knn_pca$pred_prob),
             format_results(knn_pca$pred_class, knn_pca$pred_prob, y_true))
  )
)

class_results <- bind_rows(
  lapply(names(results), function(method) {
    bind_rows(
      lapply(c("Raw", "FPCA"), function(dt) {
```

```
        m <- results[[method]][[dt]]$metrics
        data.frame(
          Method = method, Data = dt,
          Accuracy = m$Accuracy, Balanced_Accuracy = m$Balanced_Accuracy,
          Sensitivity = m$Sensitivity, Specificity = m$Specificity,
          Precision = m$Precision, F1 = m$F1,
          MCC = m$MCC, AUC = m$AUC, check.names = FALSE
        )
      })
    )
  })
)

# ================================================================
# 2f. Champion Selection
# ================================================================

# ---- Classification Champion ----
top_class <- class_results |>
  filter(Accuracy == max(Accuracy)) |>
  filter(AUC == max(AUC)) |>
  filter(MCC == max(MCC))

# Create combined method name with per-method feature annotation
class_champ_label <- paste(
  sapply(unique(top_class$Method), function(m) {
    data_used <- unique(top_class$Data[top_class$Method == m])[1]
    paste0(m, " (", data_used, ")")
  }),
  collapse = " / "
)
# For prediction, we use the Data from the first tied model (assumes same confusion matrix)
class_champ_data <- unique(top_class$Data)[1]
first_class_method <- top_class$Method[1]
class_champ_pred <- results[[first_class_method]][[class_champ_data]]$pred

# Build champion row for later display
class_champion <- data.frame(
  Method = class_champ_label,
  Data = class_champ_data,
  Accuracy = top_class$Accuracy[1],
  AUC = top_class$AUC[1],
```

```
  MCC = top_class$MCC[1],
  Balanced_Accuracy = top_class$Balanced_Accuracy[1],
  Sensitivity = top_class$Sensitivity[1],
  Specificity = top_class$Specificity[1],
  Precision = top_class$Precision[1],
  F1 = top_class$F1[1]
)

# ---- Clustering Champion ----
top_cluster <- cluster_summary |>
  filter(ARI == max(ARI)) |>
  filter(`Max Silhouette` == max(`Max Silhouette`))

# Create combined method name with per-method feature annotation
cluster_champ_label <- paste(
  sapply(unique(top_cluster$Method), function(m) {
    data_used <- unique(top_cluster$Data[top_cluster$Method == m])[1]
    paste0(m, " (", data_used, ")")
  }),
  collapse = " / "
)
cluster_champ_data <- unique(top_cluster$Data)[1]

# Retrieve cluster labels from the first tied method
first_cluster_method <- top_cluster$Method[1]
if (grepl("K-means", first_cluster_method)) {
  if (cluster_champ_data == "Raw") {
    cluster_labels <- km_raw$clusters
  } else {
    cluster_labels <- km_pca$clusters
  }
} else {  # Hierarchical
  if (cluster_champ_data == "Raw") {
    cluster_labels <- hc_raw$clusters
  } else {
    cluster_labels <- hc_pca$clusters
  }
}

cluster_champion <- data.frame(
  Method = cluster_champ_label,
  Data = cluster_champ_data,
```

```
  ARI = top_cluster$ARI[1],
  Max_Silhouette = top_cluster$`Max Silhouette`[1]
)

# ---- Tables of metrics ----
class_results_core <- class_results |>
  select(Method, Data, Accuracy, Sensitivity, Specificity, AUC)

class_results_extra <- class_results |>
  select(Method, Data, Balanced_Accuracy, Precision, F1, MCC)

knitr::kable(class_results_core,
             # caption = "Core Classification Metrics (LOOCV)",
             format = "latex", booktabs = TRUE,
             escape = TRUE, digits = 3)
```

Method	Data	Accuracy	Sensitivity	Specificity	AUC
SVM	Raw	0.882	0.889	0.875	0.903
SVM	FPCA	0.765	0.778	0.750	0.833
RF	Raw	0.882	0.889	0.875	0.896
RF	FPCA	0.882	0.889	0.875	0.917
KNN	Raw	0.882	0.889	0.875	0.861
KNN	FPCA	0.882	0.889	0.875	0.938

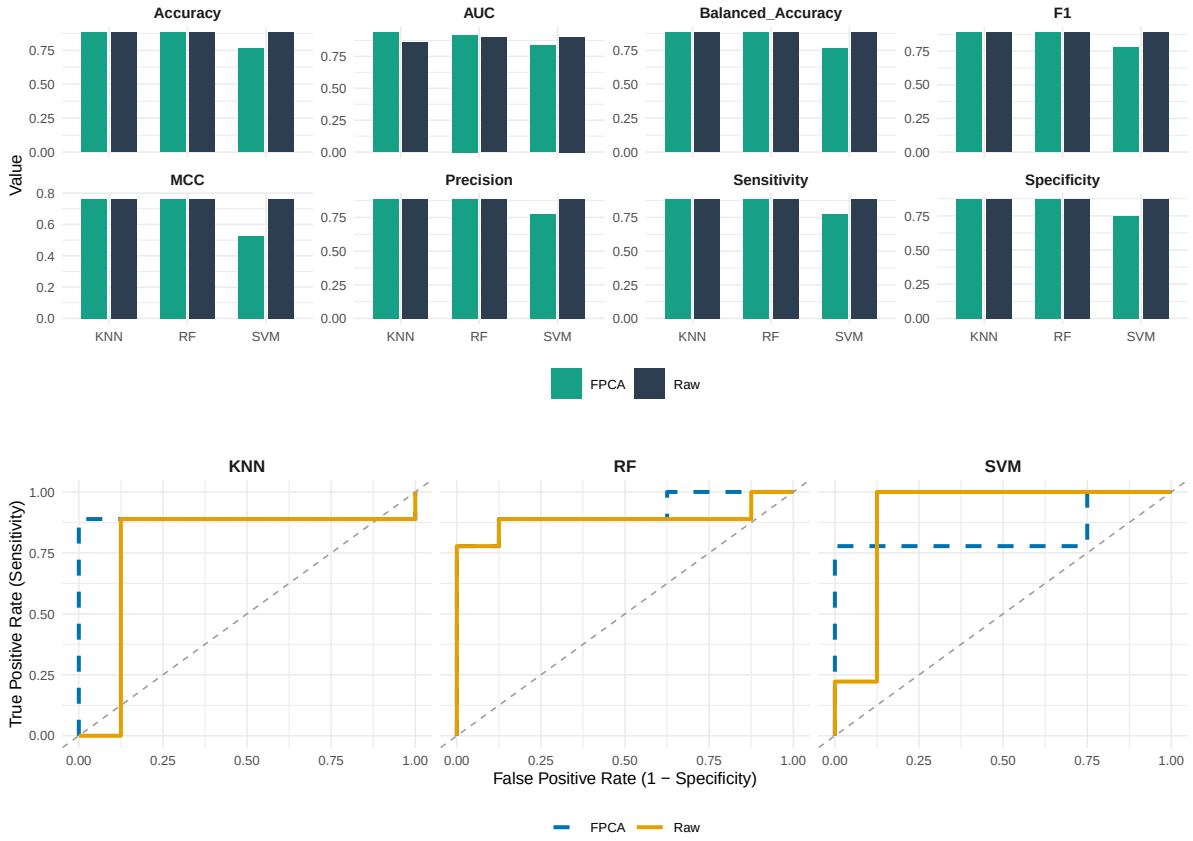
```
knitr::kable(class_results_extra,
           #   caption = "Additional Classification Metrics (LOOCV)",
             format = "latex", booktabs = TRUE,
             escape = TRUE, digits = 3)
```

Method	Data	Balanced_Accuracy	Precision	F1	MCC
SVM	Raw	0.882	0.889	0.889	0.764
SVM	FPCA	0.764	0.778	0.778	0.528
RF	Raw	0.882	0.889	0.889	0.764
RF	FPCA	0.882	0.889	0.889	0.764
KNN	Raw	0.882	0.889	0.889	0.764
KNN	FPCA	0.882	0.889	0.889	0.764

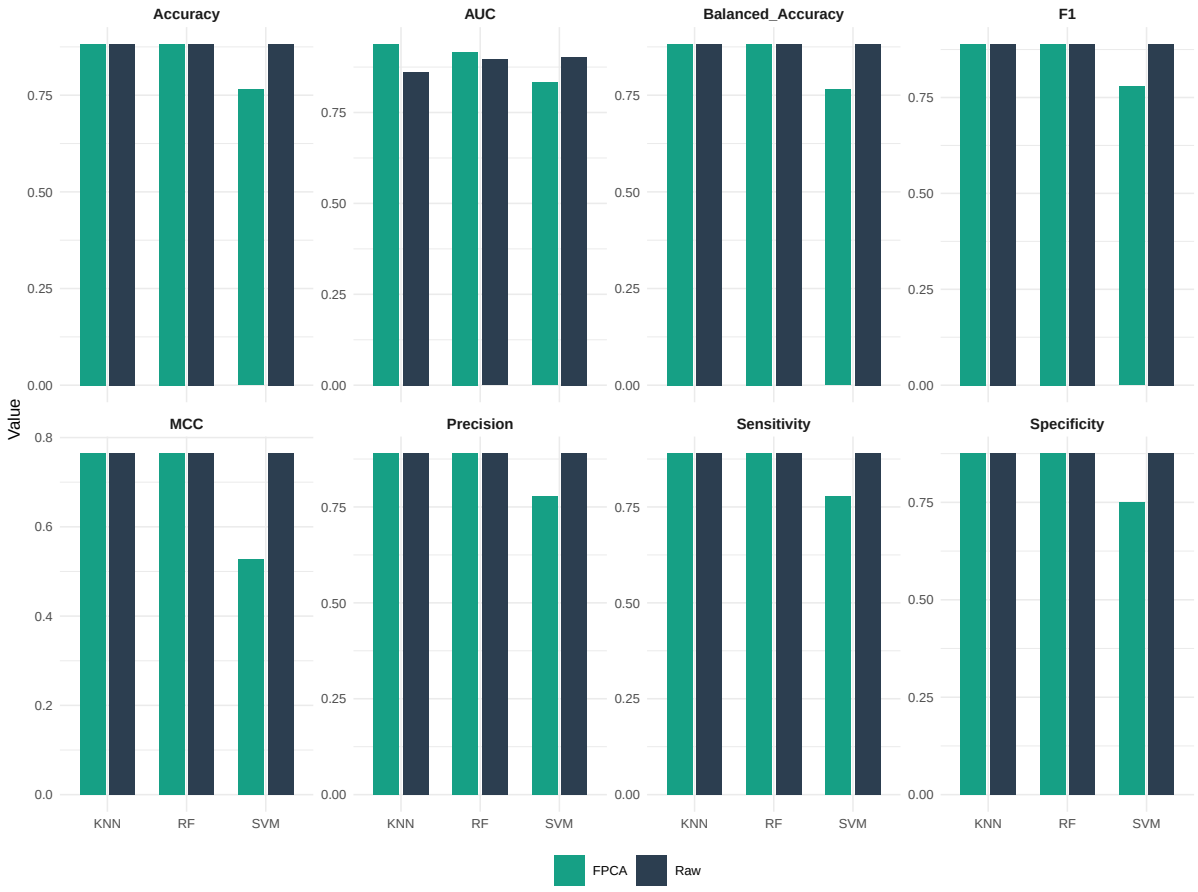
```
# ---- Metric bar plot & ROC curves ----
class_long <- class_results |>
```

```
  pivot_longer(
    cols = c(Accuracy, Balanced_Accuracy, Sensitivity,
             Specificity, Precision, F1, MCC, AUC),
    names_to = "Metric", values_to = "Value"
  )

p_class_bar <- ggplot(class_long, aes(x = Method, y = Value, fill = Data)) +
  geom_bar(stat = "identity", position = position_dodge(0.75), width = 0.65) +
  facet_wrap(~Metric, scales = "free_y", nrow = 2) +
  labs(title = " ") + #Classification Performance Breakdown Across Metrics
  scale_fill_manual(values = c(Raw = "#2C3E50", FPCA = "#16A085")) +
  theme_minimal(base_size = 8.5) +
  theme(plot.title = element_text(face = "bold", size = 10),
        legend.position = "bottom", legend.title = element_blank(),
        strip.text = element_text(face = "bold", size = 8),
        axis.title.x = element_blank())

roc_data_faceted <- bind_rows(
  lapply(names(results), function(method) {
    bind_rows(
      lapply(c("Raw", "FPCA"), function(dt) {
        roc_obj <- results[[method]][[dt]]$roc_obj
        auc_val <- results[[method]][[dt]]$metrics$AUC
        data.frame(
          Method = method,
          Data = dt,
          TPR = roc_obj$sensitivities,
          FPR = 1 - roc_obj$specificities,
          AUC = round(auc_val, 3)
        )
      })
    )
  })
) |>
  arrange(Method, Data, FPR, TPR)

p_roc_faceted <- ggplot(roc_data_faceted,
                        aes(x = FPR, y = TPR, color = Data, linetype = Data)) +
  geom_abline(slope = 1, intercept = 0, color = "grey60", linetype = "dashed") +
  geom_step(linewidth = 1) +
  facet_wrap(~ Method, ncol = 3) +
  scale_color_manual(values = c(Raw = "#E69F00", FPCA = "#0072B2")) +
```

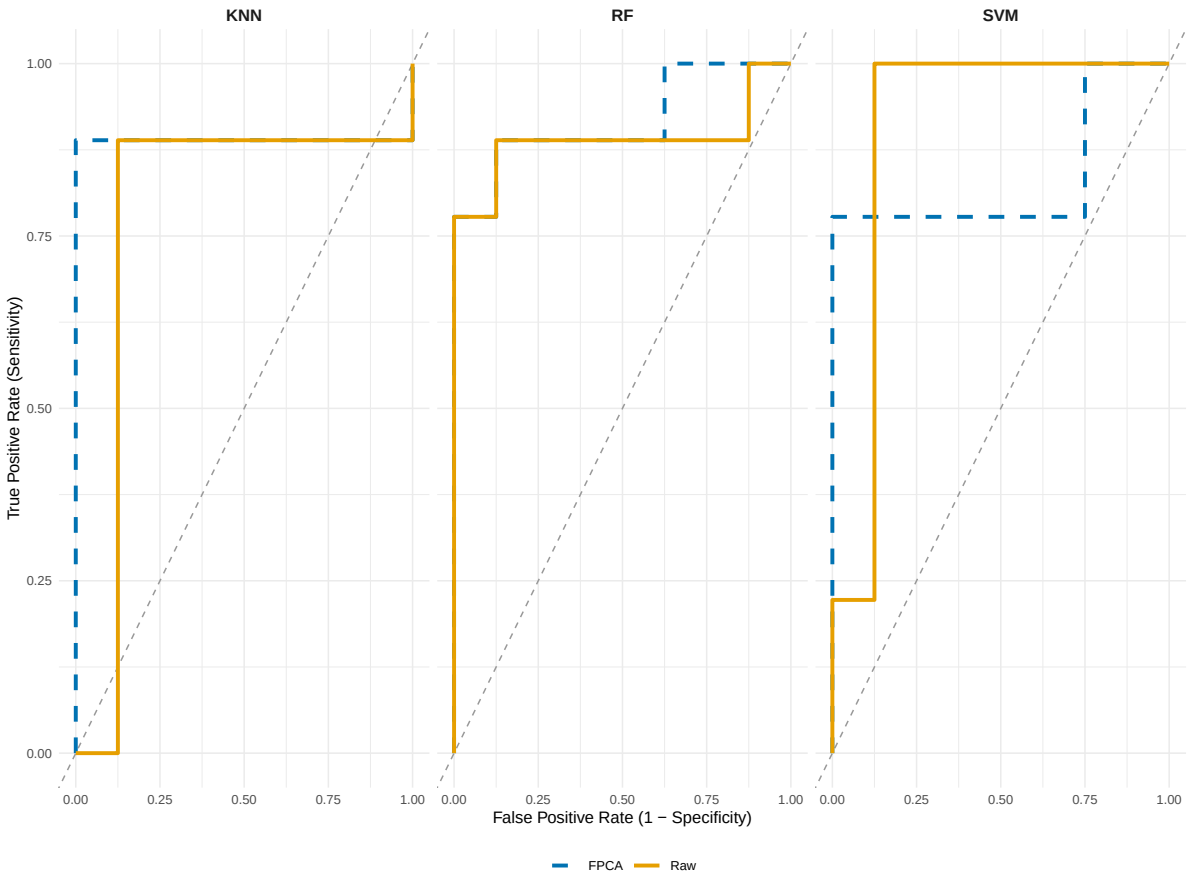
```
  scale_linetype_manual(values = c(Raw = "solid", FPCA = "dashed")) +
  labs(title = " ", # ROC Curves by Algorithm (Faceted, Stepwise)
       x = "False Positive Rate (1 - Specificity)",
       y = "True Positive Rate (Sensitivity)") +
  theme_minimal(base_size = 8.5) +
  theme(plot.title = element_text(face = "bold", size = 10),
        legend.position = "bottom",
        legend.title = element_blank(),
        strip.text = element_text(face = "bold", size = 9),
        legend.text = element_text(size = 7))

eval_grid <- (wrap_plots(p_class_bar, p_roc_faceted, ncol = 1, heights = c(1, 0.9)) &
                plot_annotation(title = " ") & #Classification Performance Evaluation
                theme(plot.title = element_text(face = "bold", size = 11)))
print(eval_grid)
```



```
print(p_class_bar)
```




```
print(p_roc_faceted)
```




```
# ======================================================================
# 3. Executive Dashboard
# ======================================================================

# 3.1 Confusion Matrix for Champion Classifier
cm <- table(
  Pred = factor(ifelse(class_champ_pred == 0, "Asymp.", "Symp."),
                levels = c("Asymp.", "Symp.")),
  True = factor(ifelse(y_true == 0, "Asymp.", "Symp."),
                levels = c("Asymp.", "Symp."))
)
cm_df <- as.data.frame(cm)

p_cm_class <- ggplot(cm_df, aes(x = True, y = Pred, fill = Freq)) +
  geom_tile(color = "white", linewidth = 0.8) +
  geom_text(aes(label = Freq), size = 8, fontface = "bold") +
  scale_fill_gradient(low = "#EAECEE", high = "#2C3E50") +
```

```
  labs(#title = paste("Supervised Classification Champion CM (", class_champion$Method, "- "
                      #class_champion$Data, ")"),
       #subtitle = paste("Top Accuracy =", round(class_champion$Accuracy, 3),
                        #"| AUC =", round(class_champion$AUC, 3)),
       x = "True Label", y = "Predicted Label") +
  theme_minimal(base_size = 8.5) +
  theme(plot.title = element_text(face = "bold", size = 9.5),
        plot.subtitle = element_text(size = 8, color = "#B03A2E", face = "bold"),
        legend.position = "none",
        plot.margin = ggplot2::margin(2, 2, 2, 2, "mm"))

# 3.2 Confusion Matrix for Champion Clusterer
cm_cluster <- table(Pred = factor(cluster_labels, levels = 1:max(cluster_labels)),
                    True = factor(y_true, levels = c(0, 1)))
cm_cluster_df <- as.data.frame(cm_cluster)
levels(cm_cluster_df$True) <- c("Asymp.", "Symp.")

p_cm_cluster <- ggplot(cm_cluster_df, aes(x = True, y = Pred, fill = Freq)) +
  geom_tile(color = "white", linewidth = 0.8) +
  geom_text(aes(label = Freq), size = 8, fontface = "bold") +
  scale_fill_gradient(low = "#EAECEE", high = "#2C3E50") +
  labs(#title = paste("Unsupervised Clustering Champion: ", cluster_champion$Method,
                   # " (", cluster_champion$Data, ")"),
      # subtitle = paste0("ARI = ", round(cluster_champion$ARI, 3),
                          #", Max Silhouette = ",
                          #round(cluster_champion$Max_Silhouette, 3)),
       x = "True Label", y = "Cluster ID") +
  theme_minimal(base_size = 8.5) +
  theme(plot.title = element_text(face = "bold", size = 9.5),
        plot.subtitle = element_text(size = 8, color = "#B03A2E", face = "bold"),
        legend.position = "none",
        plot.margin = ggplot2::margin(2, 2, 2, 2, "mm"))

# 3.3 Champion Summary Table
champ_data <- data.frame(
  Task = c("Unsupervised Clustering", "Supervised Classification"),
  Best_Model = c(
    cluster_champion$Method,
    class_champion$Method
  ),
  Key_Metric_1 = c(
    paste0("Max Silhouette: ", round(cluster_champion$Max_Silhouette, 3)),
```

```
    paste0("Accuracy: ", round(class_champion$Accuracy, 3))
  ),
  Key_Metric_2 = c(
    paste0("ARI (vs True Labels): ", round(cluster_champion$ARI, 3)),
    paste0("AUC: ", round(class_champion$AUC, 3))
  )
) |>
  mutate(across(where(is.character), sanitize_latex))

table_plot <- ggplot(champ_data, aes(y = rev(seq_len(nrow(champ_data))))) +
  annotate("text", x = 0.0, y = nrow(champ_data) + 0.5,
           label = "Task", hjust = 0, fontface = "bold", size = 3.2) +
  annotate("text", x = 1.3, y = nrow(champ_data) + 0.5,
           label = "Best Model", hjust = 0, fontface = "bold", size = 3.2) +
  annotate("text", x = 2.8, y = nrow(champ_data) + 0.5,
           label = "Key Metric 1", hjust = 0, fontface = "bold", size = 3.2) +
  annotate("text", x = 4.1, y = nrow(champ_data) + 0.5,
           label = "Key Metric 2", hjust = 0, fontface = "bold", size = 3.2) +
  geom_text(aes(x = 0.0, label = Task), hjust = 0, vjust = 0.5,
            size = 3.0, fontface = "bold") +
  geom_text(aes(x = 1.3, label = Best_Model), hjust = 0, vjust = 0.5, size = 3.0) +
  geom_text(aes(x = 2.8, label = Key_Metric_1), hjust = 0, vjust = 0.5, size = 3.0) +
  geom_text(aes(x = 4.1, label = Key_Metric_2), hjust = 0, vjust = 0.5, size = 3.0) +
  geom_hline(yintercept = nrow(champ_data) + 0.3,
             linetype = "solid", color = "black", linewidth = 0.5) +
  geom_hline(yintercept = 0.5, linetype = "solid", color = "black", linewidth = 0.5) +
  xlim(-0.1, 5.2) + ylim(0.3, nrow(champ_data) + 0.8) +
  theme_void() +
  theme(plot.margin = ggplot2::margin(2, 2, 2, 2, "mm"))

# 3.4 Raw vs FPCA Performance Comparison
cluster_best_raw <- cluster_summary |>
  filter(Data == "Raw") |>
  slice_max(ARI, with_ties = FALSE)

cluster_best_fpca <- cluster_summary |>
  filter(Data == "FPCA") |>
  slice_max(ARI, with_ties = FALSE)

class_best_raw <- class_results |>
  filter(Data == "Raw") |>
  arrange(desc(Accuracy), desc(AUC), desc(MCC)) |>
```

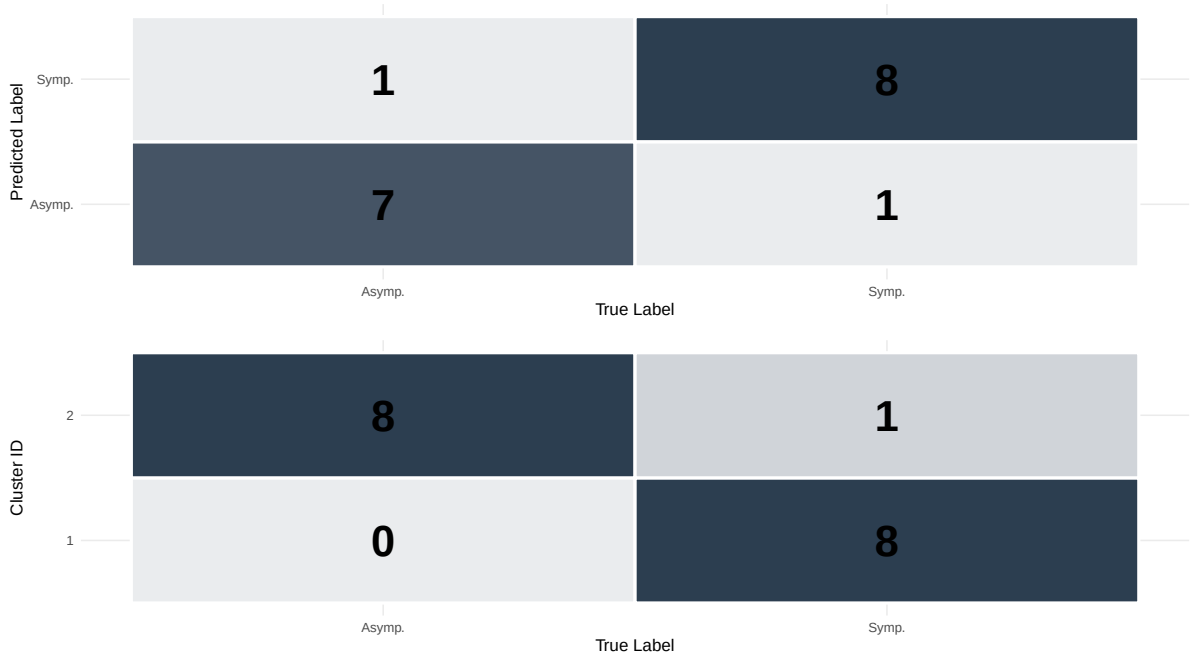
```
  slice(1)

class_best_fpca <- class_results |>
  filter(Data == "FPCA") |>
  arrange(desc(Accuracy), desc(AUC), desc(MCC)) |>
  slice(1)

comparison_table <- data.frame(
  Metric = c("Clustering ARI", "Clustering Silhouette",
             "Classification Accuracy", "Classification AUC",
             "Classification F1"),
  Raw = c(cluster_best_raw$ARI, cluster_best_raw$`Max Silhouette`,
          class_best_raw$Accuracy, class_best_raw$AUC, class_best_raw$F1),
  FPCA = c(cluster_best_fpca$ARI, cluster_best_fpca$`Max Silhouette`,
           class_best_fpca$Accuracy, class_best_fpca$AUC, class_best_fpca$F1)
)

knitr::kable(comparison_table,
           #   caption = "Raw Persistence Landscapes vs. FPCA Scores Performance Comparison"
             booktabs = TRUE, digits = 3)
```

Metric	Raw	FPCA
Clustering ARI	0.764	0.764
Clustering Silhouette	0.482	0.517
Classification Accuracy	0.882	0.882
Classification AUC	0.903	0.938
Classification F1	0.889	0.889

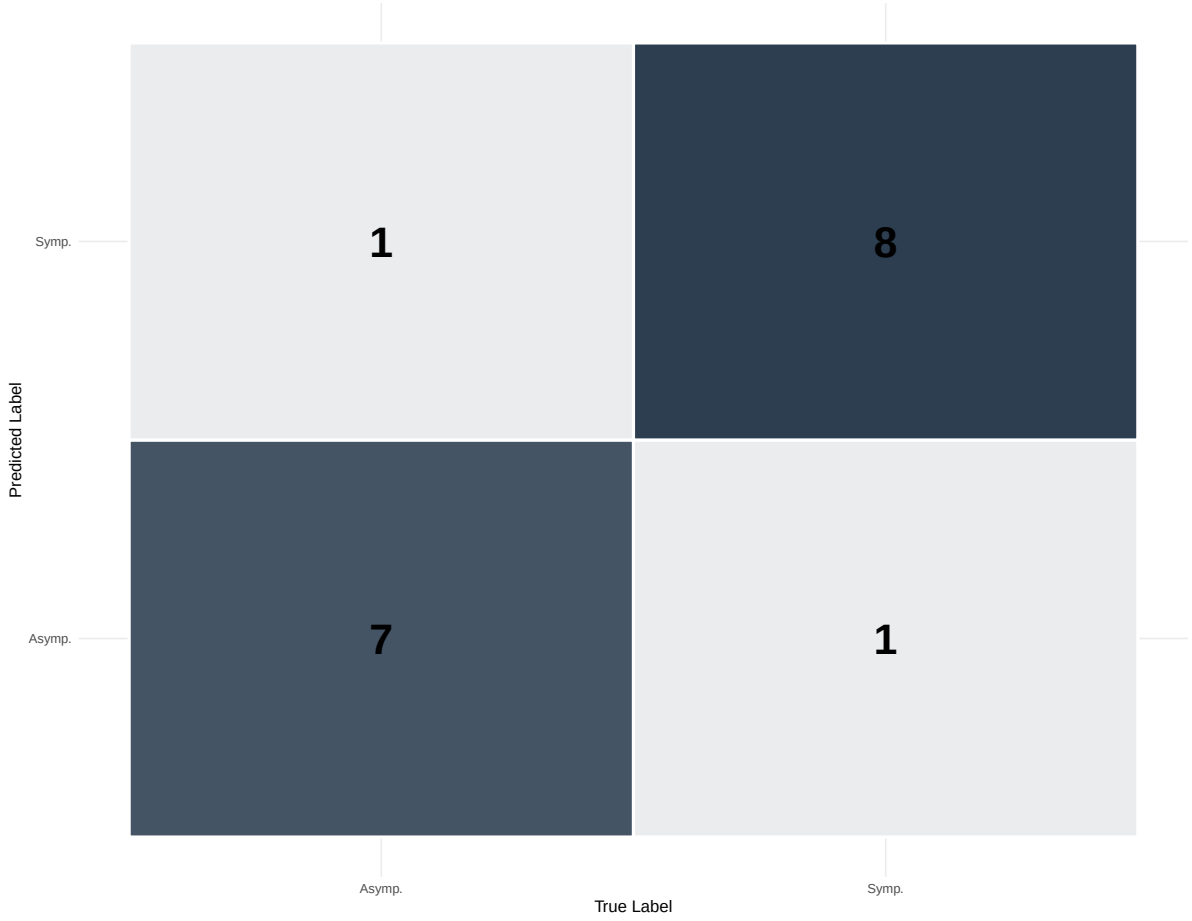
```
# Dashboard assembly
summary_dashboard <- (wrap_plots(
  wrap_elements(table_plot), p_cm_class,
  p_cm_cluster, ncol = 1,
  heights = c(0.30, 0.35, 0.35)
) &
 # plot_annotation(
  #  title = "Machine Learning Pipeline: Dual Champion Executive Dashboard",
  #  subtitle = "Note: Clustering and Classification are evaluated independently (Unsupervise
  #) &
  theme(
    plot.title = element_text(face = "bold", size = 11, hjust = 0.5),
```

```
    plot.subtitle = element_text(face = "italic", size = 8, color = "grey30", hjust = 0.5)
  ))
print(summary_dashboard)
```

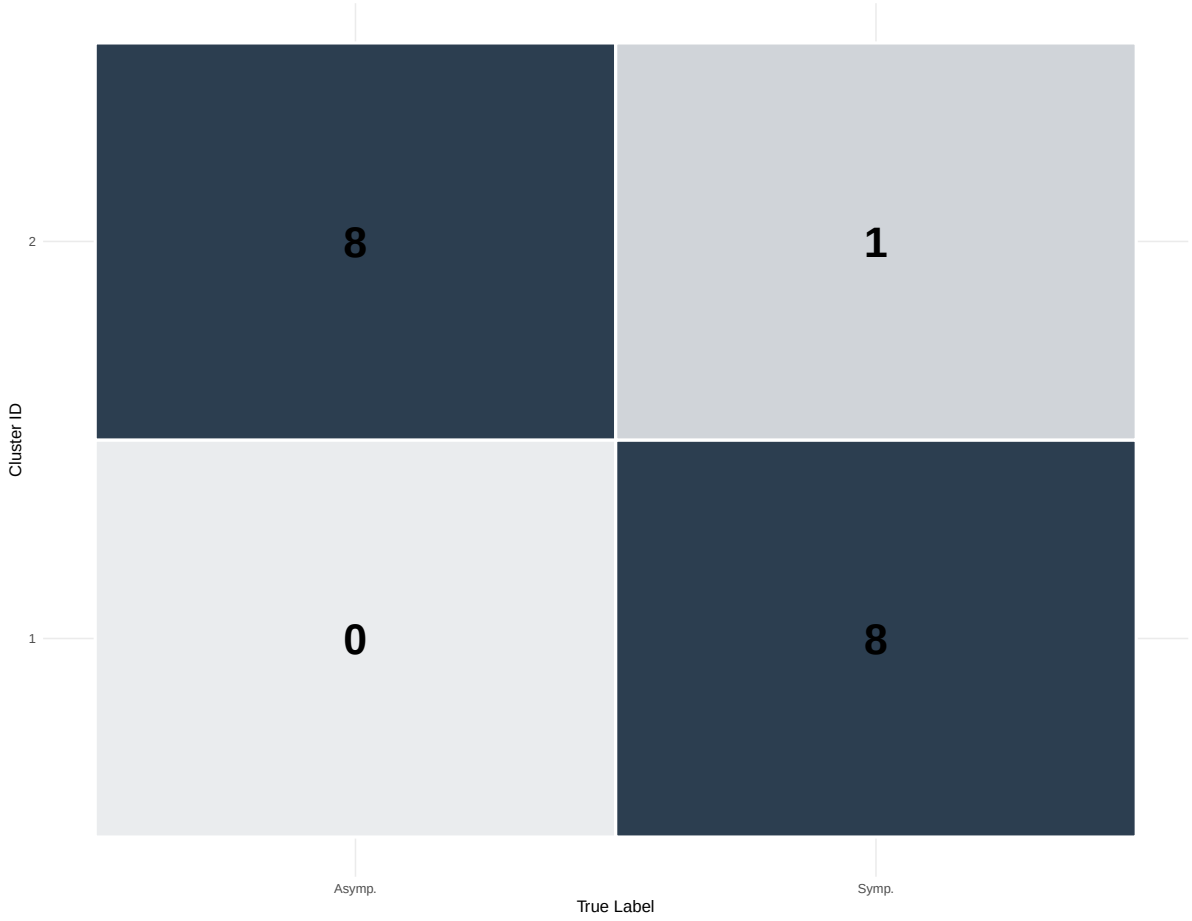
Task	Best Model	Key Metric 1	Key Metric 2
Unsupervised Clustering	K−means (FPCA) / Hierarchical (FPCA)	Max Silhouette: 0.517	ARI (vs True Labels): 0.764
Supervised Classification	KNN (FPCA)	Accuracy: 0.882	AUC: 0.938




```
print(p_cm_class)
```




```
print(p_cm_cluster)
```




```
# 3.5 Best Model Selection by Feature
cluster_raw_best <- cluster_summary |>
  filter(Data == "Raw") |>
  slice_max(ARI, with_ties = FALSE)

cluster_fpca_best <- cluster_summary |>
  filter(Data == "FPCA") |>
  slice_max(ARI, with_ties = FALSE)

class_raw_best <- class_results |>
  filter(Data == "Raw") |>
  arrange(desc(Accuracy), desc(AUC)) |>
  slice(1)

class_fpca_best <- class_results |>
  filter(Data == "FPCA") |>
```

```
  arrange(desc(Accuracy), desc(AUC)) |>
  slice(1)

overall_summary <- data.frame(
  Feature = c("Raw Landscape", "FPCA Scores", "Raw Landscape", "FPCA Scores"),
  Task = c("Classification", "Classification", "Clustering", "Clustering"),
  Best_Model = c(class_raw_best$Method, class_fpca_best$Method,
                 cluster_raw_best$Method, cluster_fpca_best$Method),
  Accuracy = c(round(class_raw_best$Accuracy, 3), round(class_fpca_best$Accuracy, 3), "-", "-
  AUC = c(round(class_raw_best$AUC, 3), round(class_fpca_best$AUC, 3), "-", "-"),
  ARI = c("-", "-", round(cluster_raw_best$ARI, 3), round(cluster_fpca_best$ARI, 3)),
  Silhouette = c("-", "-", round(cluster_raw_best$`Max Silhouette`, 3), round(cluster_fpca_be
)

knitr::kable(overall_summary,
             # caption = "Systematic Evaluation Summary",
             booktabs = TRUE, digits = 3, na = "-")
```

Feature	Task	Best_Model	Accuracy	AUC	ARI	Silhouette
Raw Landscape	Classification	SVM	0.882	0.903	-	-
FPCA Scores	Classification	KNN	0.882	0.938	-	-
Raw Landscape	Clustering	K-means	-	-	0.764	0.482
FPCA Scores	Clustering	K-means	-	-	0.764	0.517

```
# Feature-Model Interaction Summary
class_summary_dynamic <- class_results |>
  group_by(Method) |>
  summarise(
    Task = "Classification",
    Best_Feature = case_when(
      Accuracy[Data == "Raw"] > Accuracy[Data == "FPCA"] ~ "Raw",
      Accuracy[Data == "Raw"] < Accuracy[Data == "FPCA"] ~ "FPCA",
      Accuracy[Data == "Raw"] == Accuracy[Data == "FPCA"] &
        AUC[Data == "Raw"] > AUC[Data == "FPCA"] ~ "Raw",
      Accuracy[Data == "Raw"] == Accuracy[Data == "FPCA"] &
        AUC[Data == "Raw"] < AUC[Data == "FPCA"] ~ "FPCA",
      TRUE ~ "Tie"
    ),
```

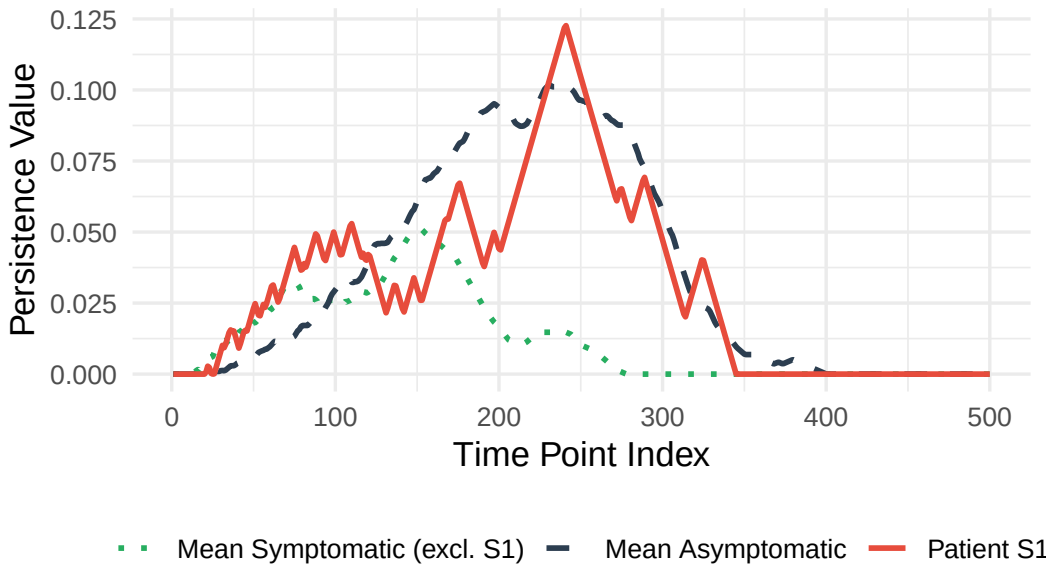
```
    Raw_Performance = paste0("Acc=", round(Accuracy[Data == "Raw"], 3),
                             ", AUC=", round(AUC[Data == "Raw"], 3)),
    FPCA_Performance = paste0("Acc=", round(Accuracy[Data == "FPCA"], 3),
                              ", AUC=", round(AUC[Data == "FPCA"], 3))
  ) |>
  ungroup()

cluster_summary_dynamic <- cluster_summary |>
  group_by(Method) |>
  summarise(
    Task = "Clustering",
    Best_Feature = case_when(
      `Max Silhouette`[Data == "Raw"] > `Max Silhouette`[Data == "FPCA"] ~ "Raw",
      `Max Silhouette`[Data == "Raw"] < `Max Silhouette`[Data == "FPCA"] ~ "FPCA",
      TRUE ~ "Tie"
    ),
    Raw_Performance = paste0("ARI=", round(ARI[Data == "Raw"], 3),
                             ", Sil=", round(`Max Silhouette`[Data == "Raw"], 3)),
    FPCA_Performance = paste0("ARI=", round(ARI[Data == "FPCA"], 3),
                              ", Sil=", round(`Max Silhouette`[Data == "FPCA"], 3))
  ) |>
  ungroup()

final_table_clean <- bind_rows(class_summary_dynamic, cluster_summary_dynamic) |>
  select(Method, Task, Best_Feature, Raw_Performance, FPCA_Performance)

knitr::kable(final_table_clean,
             #  caption = "Feature-Model Interaction Summary",
             booktabs = TRUE,
             align = c("l", "l", "l", "l", "l"))
```

Method	Task	Best_Feature	Raw_Performance	FPCA_Performance
KNN	Classification	FPCA	Acc=0.882, AUC=0.861	Acc=0.882, AUC=0.938
RF	Classification	FPCA	Acc=0.882, AUC=0.896	Acc=0.882, AUC=0.917
SVM	Classification	Raw	Acc=0.882, AUC=0.903	Acc=0.765, AUC=0.833
Hierarchical	Clustering	FPCA	ARI=0.764, Sil=0.482	ARI=0.764, Sil=0.517
K-means	Clustering	FPCA	ARI=0.764, Sil=0.482	ARI=0.764, Sil=0.517

```r
# ================================================================
# S1 Misclassification Diagnostic
# ================================================================

# Ensure DataBoth has row names for sample identification
if (is.null(rownames(DataBoth))) {
  rownames(DataBoth) <- m
}

s1_idx <- which(m == "S1")

# ---- 1. L2 distance analysis (raw feature space) ----
s1_data <- as.numeric(DataBoth["S1", ])

# Mean of asymptomatic group (all)
asymptomatic_data <- DataBoth[group == "Asymptomatic", , drop = FALSE]
mean_asymptomatic <- colMeans(asymptomatic_data)

# Mean of symptomatic group (excluding S1)
symptomatic_data <- DataBoth[group == "Symptomatic" & rownames(DataBoth) != "S1", , drop = FA
if (nrow(symptomatic_data) == 0) {
  warning("Symptomatic group contains no other samples besides S1.")
}
mean_symptomatic <- colMeans(symptomatic_data)

dist_to_asymp <- sqrt(sum((s1_data - mean_asymptomatic)^2))
dist_to_symp <- sqrt(sum((s1_data - mean_symptomatic)^2))

# L2 distance table
dist_table <- data.frame(
  Comparison = c("S1 vs Asymptomatic Mean", "S1 vs Symptomatic Mean"),
  L2_Distance = round(c(dist_to_asymp, dist_to_symp), 4)
)

knitr::kable(
  dist_table,
 # caption = "L2 Distance from S1 to Group Means (Raw Landscape Space)",
  booktabs = TRUE,
  digits = 4,
  align = "lc"
)
```

Comparison	L2_Distance
S1 vs Asymptomatic Mean	0.3852
S1 vs Symptomatic Mean	0.7891

```r
# ---- 2. Visualisation: S1 vs. class means ----
time_points <- 1:ncol(DataBoth)
plot_df <- data.frame(
  Time = rep(time_points, 3),
  Value = c(s1_data, mean_asymptomatic, mean_symptomatic),
  Curve = factor(rep(c("Patient S1", "Mean Asymptomatic", "Mean Symptomatic (excl. S1)"),
                     each = length(time_points)),
                 levels = c("Mean Symptomatic (excl. S1)", "Mean Asymptomatic", "Patient S1")
)

library(ggplot2)
p_s1_landscape <- ggplot(plot_df, aes(x = Time, y = Value, color = Curve, linetype = Curve))
  geom_line(size = 1) +
  scale_color_manual(values = c("Patient S1" = "#E74C3C",
                                "Mean Asymptomatic" = "#2C3E50",
                                "Mean Symptomatic (excl. S1)" = "#27AE60")) +
  scale_linetype_manual(values = c("Patient S1" = "solid",
                                   "Mean Asymptomatic" = "dashed",
                                   "Mean Symptomatic (excl. S1)" = "dotted")) +
  labs(title = " ", #S1 Persistence Landscape vs. Class Means
       x = "Time Point Index", y = "Persistence Value",
       #subtitle = paste0("L2 to Asymptomatic = ", round(dist_to_asymp, 4),
                         # ", L2 to Symptomatic = ", round(dist_to_symp, 4))
       )+
  theme_minimal(base_size = 13) +
  theme(legend.position = "bottom", legend.title = element_blank())

print(p_s1_landscape)
```




```
# ---- 3. Supervised learning: classifier predictions ----
cat("\n\n")
```

```
models <- list(
  "SVM_Raw"    = svm_raw$pred_class,
  "SVM_FPCA"     = svm_pca$pred_class,
  "RF_Raw"     = rf_raw$pred_class,
  "RF_FPCA"      = rf_pca$pred_class,
  "KNN_Raw"    = knn_raw$pred_class,
  "KNN_FPCA"     = knn_pca$pred_class
)

# Build summary table
results_df <- data.frame(
  Model = names(models),
  Predicted = sapply(models, function(x) x[s1_idx]),
  True = y_true[s1_idx],
  Correct = sapply(models, function(x) {
    ifelse(x[s1_idx] == y_true[s1_idx], "Yes", "No")
  })
)

knitr::kable(
  results_df,
  # caption = "S1: Supervised Classification Predictions",
```

```
  booktabs = TRUE,
  align = "lccc"
)
```

	Model	Predicted	True	Correct
SVM_Raw	SVM_Raw	0	1	No
SVM_FPCA	SVM_FPCA	0	1	No
RF_Raw	RF_Raw	0	1	No
RF_FPCA	RF_FPCA	0	1	No
KNN_Raw	KNN_Raw	0	1	No
KNN_FPCA	KNN_FPCA	0	1	No

```
# ---- 4. Unsupervised learning: cluster assignments ----

# ---- 4.1 K-means clustering ----
# Cluster composition tables
km_raw_table <- table(Cluster = km_raw$clusters, True_Label = ifelse(y_true == 1, "Symp", "As
km_pca_table <- table(Cluster = km_pca$clusters, True_Label = ifelse(y_true == 1, "Symp", "As

knitr::kable(
  km_raw_table,
 # caption = "K-means (Raw): Cluster Composition by True Label",
  booktabs = TRUE,
  align = "lcc"
)
```

Asymp	Symp
0	8
8	1

```
knitr::kable(
  km_pca_table,
 # caption = "K-means (PC): Cluster Composition by True Label",
  booktabs = TRUE,
  align = "lcc"
)
```

Asymp	Symp
0	8
8	1

```
# S1's cluster and its composition
s1_km_raw <- km_raw$clusters[s1_idx]
s1_km_pca <- km_pca$clusters[s1_idx]

s1_cluster_composition_raw <- table(ifelse(y_true[km_raw$clusters == s1_km_raw] == 1, "Symp"
s1_cluster_composition_pca <- table(ifelse(y_true[km_pca$clusters == s1_km_pca] == 1, "Symp"

km_s1_summary <- data.frame(
  Representation = c("Raw", "FPCA"),
  S1_Cluster = c(s1_km_raw, s1_km_pca),
  Asymp_in_Cluster = c(s1_cluster_composition_raw["Asymp"], s1_cluster_composition_pca["Asym
  Symp_in_Cluster = c(s1_cluster_composition_raw["Symp"], s1_cluster_composition_pca["Symp"]
)
km_s1_summary[is.na(km_s1_summary)] <- 0

knitr::kable(
  km_s1_summary,
 # caption = "K-means: S1 Cluster and Its Composition",
  booktabs = TRUE,
  align = "lccc"
)
```

Representation	S1_Cluster	Asymp_in_Cluster	Symp_in_Cluster
Raw	2	8	1
FPCA	2	8	1

```
# ---- 4.2 Hierarchical clustering ----
hc_raw_table <- table(Cluster = hc_raw$clusters, True_Label = ifelse(y_true == 1, "Symp", "As
hc_pca_table <- table(Cluster = hc_pca$clusters, True_Label = ifelse(y_true == 1, "Symp", "As

knitr::kable(
  hc_raw_table,
 # caption = "Hierarchical (Raw): Cluster Composition by True Label",
  booktabs = TRUE,
  align = "lcc"
```

```
)
```

Asymp	Symp
8	1
0	8

```
knitr::kable(
  hc_pca_table,
 # caption = "Hierarchical (PC): Cluster Composition by True Label",
  booktabs = TRUE,
  align = "lcc"
)
```

Asymp	Symp
8	1
0	8

```
# S1's cluster and its composition
s1_hc_raw <- hc_raw$clusters[s1_idx]
s1_hc_pca <- hc_pca$clusters[s1_idx]

s1_cluster_composition_hc_raw <- table(ifelse(y_true[hc_raw$clusters == s1_hc_raw] == 1, "Sy
s1_cluster_composition_hc_pca <- table(ifelse(y_true[hc_pca$clusters == s1_hc_pca] == 1, "Sy

hc_s1_summary <- data.frame(
  Representation = c("Raw", "FPCA"),
  S1_Cluster = c(s1_hc_raw, s1_hc_pca),
  Asymp_in_Cluster = c(s1_cluster_composition_hc_raw["Asymp"], s1_cluster_composition_hc_pca
  Symp_in_Cluster = c(s1_cluster_composition_hc_raw["Symp"], s1_cluster_composition_hc_pca["S
)
hc_s1_summary[is.na(hc_s1_summary)] <- 0

knitr::kable(
  hc_s1_summary,
 # caption = "Hierarchical: S1 Cluster and Its Composition",
  booktabs = TRUE,
  align = "lccc"
)
```

Representation	S1_Cluster	Asymp_in_Cluster	Symp_in_Cluster
Raw	1	8	1
FPCA	1	8	1

```
# ---- 4.3 Combined summary ----
s1_cluster_all <- rbind(
  data.frame(Method = "K-means (Raw)", km_s1_summary[1, -1]),
  data.frame(Method = "K-means (FPCA)", km_s1_summary[2, -1]),
  data.frame(Method = "Hierarchical (Raw)", hc_s1_summary[1, -1]),
  data.frame(Method = "Hierarchical (FPCA)", hc_s1_summary[2, -1])
)

knitr::kable(
  s1_cluster_all,
 # caption = "S1 Cluster Assignment Summary (All Unsupervised Methods)",
  booktabs = TRUE,
  align = "lccc"
)
```

	Method	S1_Cluster	Asymp_in_Cluster	Symp_in_Cluster
1	K-means (Raw)	2	8	1
2	K-means (FPCA)	2	8	1
11	Hierarchical (Raw)	1	8	1
21	Hierarchical (FPCA)	1	8	1

```
# ---- 5. Diagnostic conclusion ----
if (dist_to_asymp < dist_to_symp) {
  cat("\n**Diagnostic Conclusion**: S1 is mathematically closer to the **Asymptomatic** mean
This explains why the classifiers consistently mislabel S1 as Asymptomatic,
even though its ground truth is Symptomatic.\n")
} else {
  cat("\n**Diagnostic Conclusion**: S1 is closer to the **Symptomatic** mean.
Misclassification may be due to other factors (e.g., boundary overlap or noise).\n")
}
```

```
**Diagnostic Conclusion**: S1 is mathematically closer to the **Asymptomatic** mean.
This explains why the classifiers consistently mislabel S1 as Asymptomatic,
even though its ground truth is Symptomatic.
```

R output

```
# ================================================================
# 1. Create output folders
# ================================================================
fig_dir <- "../../output/R_figures"
tab_dir <- "../../output/R_tables"

dir.create(fig_dir, recursive = TRUE, showWarnings = FALSE)
dir.create(tab_dir, recursive = TRUE, showWarnings = FALSE)

# ================================================================
# 2. Export all plots (one PDF per print() statement)
# ================================================================

# Collect all plot objects
all_plots <- list(

  # ---- funTDA ----
  pca_plot = pca_plot,

  # ---- Clustering: Parameter diagnostics ----
  diag_grid = diag_grid,

  # ---- Clustering: K-means (optimal k) ----
  p_gt_cluster = p_gt_cluster,
  p_km_raw = p_km_raw,
  p_km_pca = p_km_pca,
  km_scatter_panel = km_scatter_panel,

  # ---- Clustering: K-means (k=3 manual inspection) ----
  p_km_raw_k3 = p_km_raw_k3,
  p_km_pca_k3 = p_km_pca_k3,
  km3_scatter_panel = km3_scatter_panel,

  # ---- Clustering: Hierarchical dendrograms ----
  dendro_2x4_grid = dendro_2x4_grid,

  # ---- Clustering: Hierarchical scatter ----
  p_hc_raw = p_hc_raw,
  p_hc_pca = p_hc_pca,
  hc_scatter_panel = hc_scatter_panel,
```

```
  # ---- Classification: SVM decision boundaries ----
  p_svm_boundaries = p_svm_boundaries,

  # ---- Classification: RF variable importance ----
  p_rf_imp = p_rf_imp,

  # ---- Classification: Hyperparameter trends ----
  p_knn_trend = p_knn_trend,
  p_svm_trend = p_svm_trend,
  hyperparam_grid = hyperparam_grid,

  # ---- Classification: SVM predictions ----
  p_gt_class = p_gt_class,
  p_svm_raw = p_svm_raw,
  p_svm_pca = p_svm_pca,
  svm_grid = svm_grid,

  # ---- Classification: RF predictions ----
  p_rf_raw = p_rf_raw,
  p_rf_pca = p_rf_pca,
  rf_grid = rf_grid,

  # ---- Classification: KNN predictions ----
  p_knn_raw = p_knn_raw,
  p_knn_pca = p_knn_pca,
  knn_grid = knn_grid,

  # ---- Classification: Performance evaluation (bar plot + ROC) ----
  p_class_bar = p_class_bar,
  p_roc_faceted = p_roc_faceted,
  eval_grid = eval_grid,

  # ---- Executive dashboard (confusion matrices + summary table) ----
  p_cm_class = p_cm_class,
  p_cm_cluster = p_cm_cluster,
  summary_dashboard = summary_dashboard,

  # ---- S1 Diagnostic ----
  p_s1_landscape = p_s1_landscape
)

# Batch export PDFs to output/R_figures/
```

```r
purrr::iwalk(all_plots, function(p, nm) {
  if (!is.null(p) && inherits(p, c("gg", "patchwork"))) {
    ggsave(
      filename = file.path(fig_dir, paste0(nm, ".pdf")),
      plot = p,
      width = 10,
      height = 6,
      units = "in",
      create.dir = TRUE
    )
    message("Exported plot: ", fig_dir, "/", nm, ".pdf")
  } else {
    message("Skipped: ", nm, " (object is NULL or not a ggplot/patchwork)")
  }
})

# ---- Export base R plots (mean.fd) separately ----
pdf(file.path(fig_dir, "mean_symptomatic_landscape.pdf"), width = 6, height = 4)
plot(mean.fd(fdSymp), main = "")
```

```
[1] "done"
```

```r
dev.off()
```

```
pdf
  2
```

```r
message("Exported plot: ", fig_dir, "/mean_symptomatic_landscape.pdf")

pdf(file.path(fig_dir, "mean_asymptomatic_landscape.pdf"), width = 6, height = 4)
plot(mean.fd(fdAsymp), main = "")
```

```
[1] "done"
```

```r
dev.off()
```

```
pdf
  2
```

```r
message("Exported plot: ", fig_dir, "/mean_asymptomatic_landscape.pdf")


# ================================================================
# 3. Export all tables as .tex files (to output/R_tables/)
# ================================================================

tables_to_export <- list(
  # Feature extraction
  fpc_table = fpc_table,

  # Clustering summary
  cluster_summary = cluster_summary,

  # Core classification metrics
  class_results_core = class_results_core,
  class_results_extra = class_results_extra,

  # Raw vs FPCA comparison
  comparison_table = comparison_table,

  # Systematic evaluation summary
  overall_summary = overall_summary,

  # Feature-model interaction
  final_table_clean = final_table_clean,

  # S1 diagnostics
  dist_table = dist_table,
  results_df = results_df,

  # S1 cluster composition
  km_raw_table = km_raw_table,
  km_pca_table = km_pca_table,
  km_s1_summary = km_s1_summary,
  hc_raw_table = hc_raw_table,
  hc_pca_table = hc_pca_table,
  hc_s1_summary = hc_s1_summary,
  s1_cluster_all = s1_cluster_all
)

# Batch export tables as LaTeX code (without caption)
```

```r
for (nm in names(tables_to_export)) {
  df <- tables_to_export[[nm]]
  if (!is.null(df)) {
    latex_code <- knitr::kable(
      df,
      format = "latex",
      booktabs = TRUE,
      escape = TRUE,
      digits = 3
    )
    cat(latex_code, file = file.path(tab_dir, paste0(nm, ".tex")), sep = "\n")
    message("Exported table: ", tab_dir, "/", nm, ".tex")
  } else {
    message("Skipped table: ", nm, " (object is NULL)")
  }
}

message("\n Export complete! Plots are in '", fig_dir, "', tables are in '", tab_dir, "'.")
```